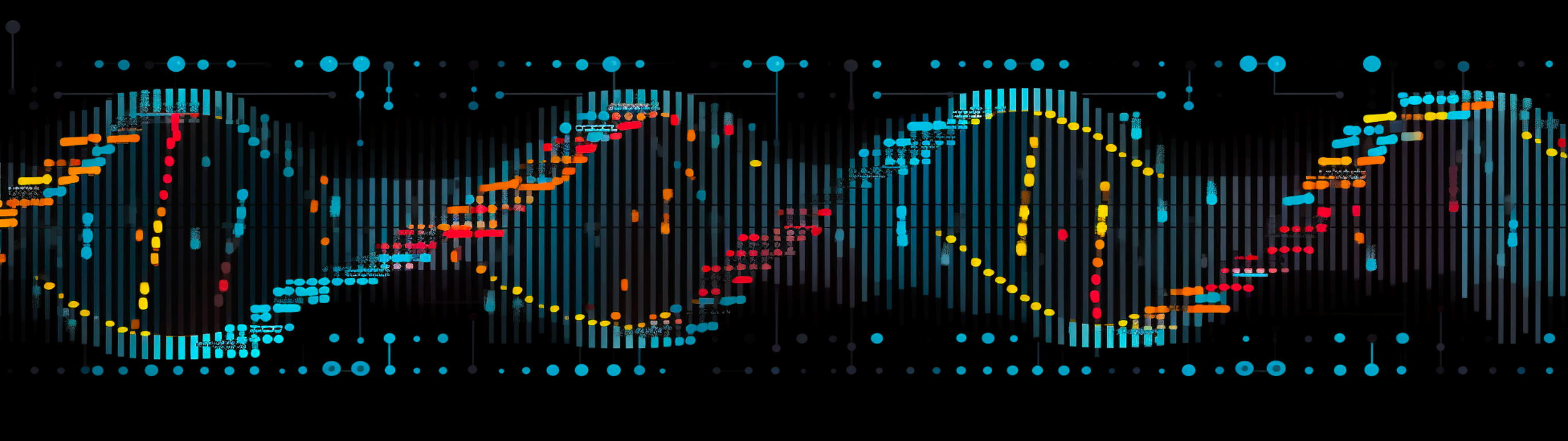




Introduction to bioinformatics

Chapter4 Machine learning in bioinformatics

- Network and clustering - Part2 (Network construction)

Jiaxing Chen

Outline

- **Network construction**
 - GRN inference from Microarray
 - GRN inference from Single cell transcript
 - Cell Cell Interaction

Network reconstruction

Use gene regulatory network reconstruction as example

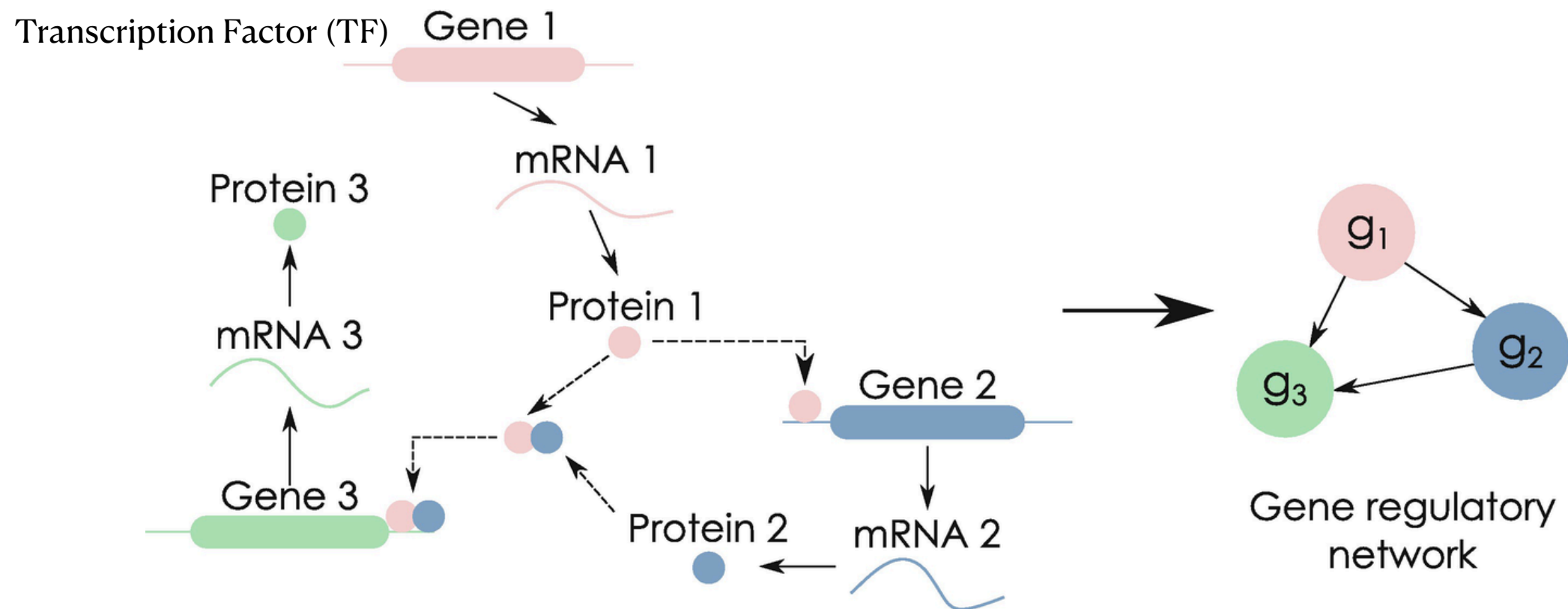
Some review:

[1] Marbach D, Costello J C, Küffner R, et al. Wisdom of crowds for robust gene network inference[J]. Nature methods, 2012, 9(8): 796-804.

[2] Pratapa A, Jaliha A P, Law J N, et al. Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data[J]. Nature methods, 2020, 17(2):

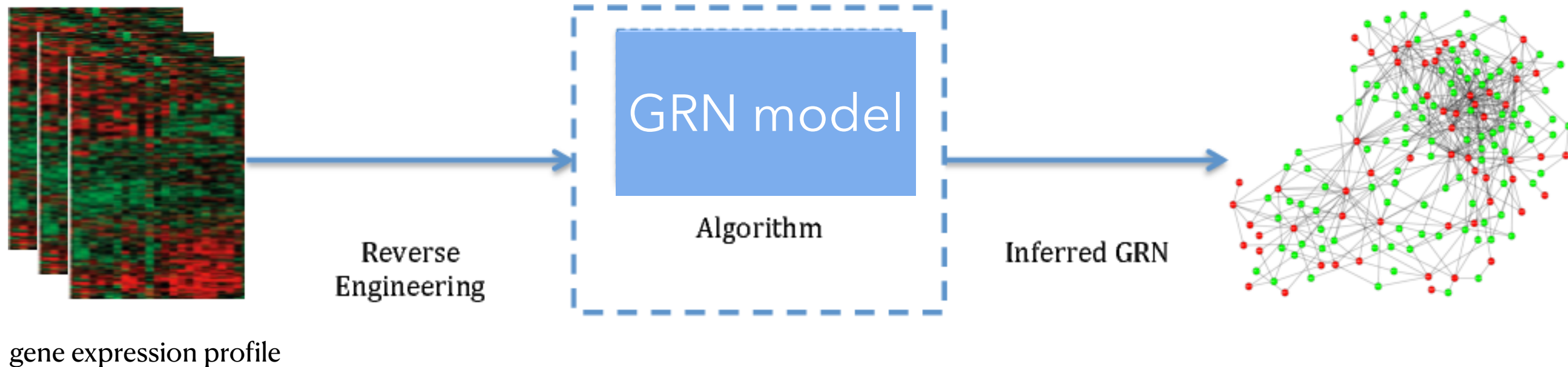
[3] Zhao M, He W, Tang J, et al. A comprehensive overview and critical evaluation of gene regulatory network inference technologies[J]. Briefings in bioinformatics, 2021, 22(5): bbab009. 147-154.

Gene regulatory network (GRN)

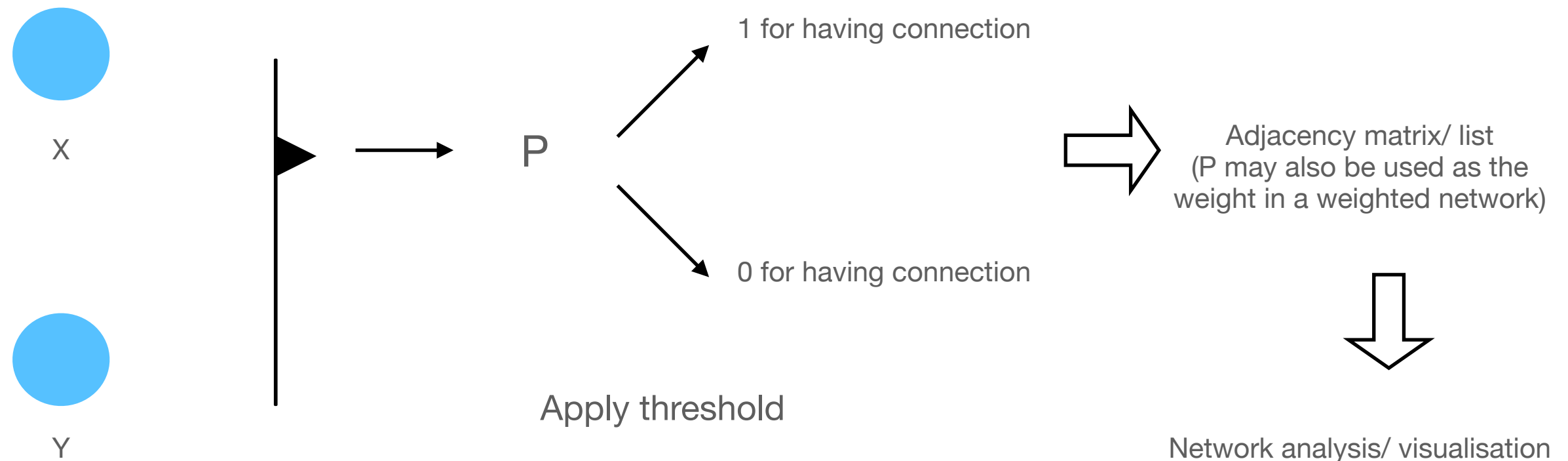


Reconstruct GRN from gene expression

Reverse Engineering

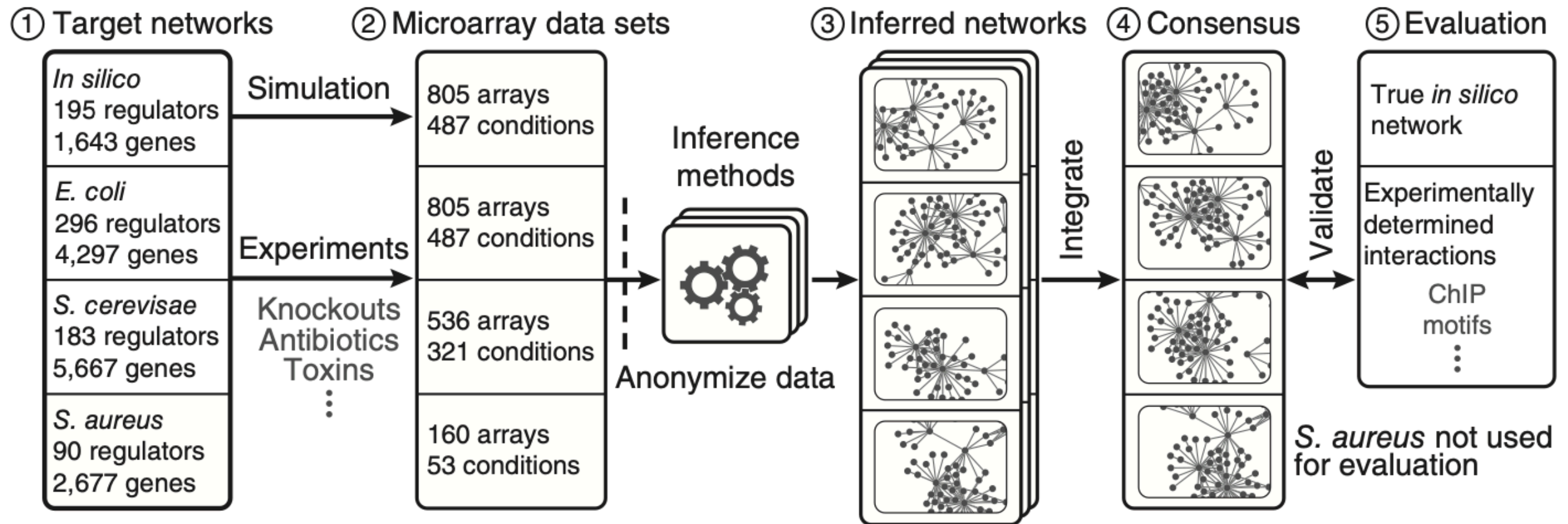


For every pair of node:



Consider the node features, to predict the probability P , that whether the two node exist edge between them.

gene regulatory network inference from Microarray



The DREAM5 network inference challenge.

Table 1 | Network inference methods

ID	Synopsis	Reference
Regression: transcription factors are selected by target gene-specific (i) sparse linear-regression and (ii) data-resampling approaches.		
1	Trustful Inference of Gene REgulation using Stability Selection (TIGRESS): (i) Lasso; (ii) the regularization parameter selects five transcription factors per target gene in each bootstrap sample.	33 ^a
2	(i) Steady-state and time-series data are combined by group Lasso; (ii) bootstrapping.	34 ^a
3	Combination of Lasso and Bayesian linear regression models learned using reversible-jump Markov chain Monte Carlo simulations.	35 ^a
4	(i) Lasso; (i) bootstrapping.	36
5	(i) Lasso; (ii) area under the stability selection curve.	36
6	Application of the Lasso toolbox GENLAB using standard parameters.	37
7	Lasso models are combined by the maximum regularization parameter selecting a given edge for the first time.	36 ^a
8	Linear regression determines the contribution of transcription factors to the expression of target genes.	— ^{a,b}
Mutual information: edges are (i) ranked based on variants of mutual information and (ii) filtered for causal relationships.		
1	Context likelihood of relatedness (CLR): (i) spline estimation of mutual information; (ii) the likelihood of each mutual information score is computed based on its local network context.	11 ^{a,b}
2	(i) Mutual information is computed from discretized expression values.	38 ^{a,b}
3	Algorithm for the reconstruction of accurate cellular networks (ARACNE): (i) kernel estimation of mutual information; (ii) the data processing inequality is used to identify direct interactions.	9 ^{a,b}
4	(i) Fast kernel-based estimation of mutual information; (ii) Bayesian local causal discovery (BLCD) and Markov blanket (HITON-PC) algorithm to identify direct interactions.	39 ^a
5	(i) Mutual information and Pearson’s correlation are combined; (ii) BLCD and HITON-PC algorithm.	39 ^a
Correlation: edges are ranked based on variants of correlation.		
1	Absolute value of Pearson’s correlation coefficient.	38
2	Signed value of Pearson’s correlation coefficient.	38 ^{a,b}
3	Signed value of Spearman’s correlation coefficient.	38 ^{a,b}

Bayesian networks: optimize posterior probabilities by different heuristic searches.

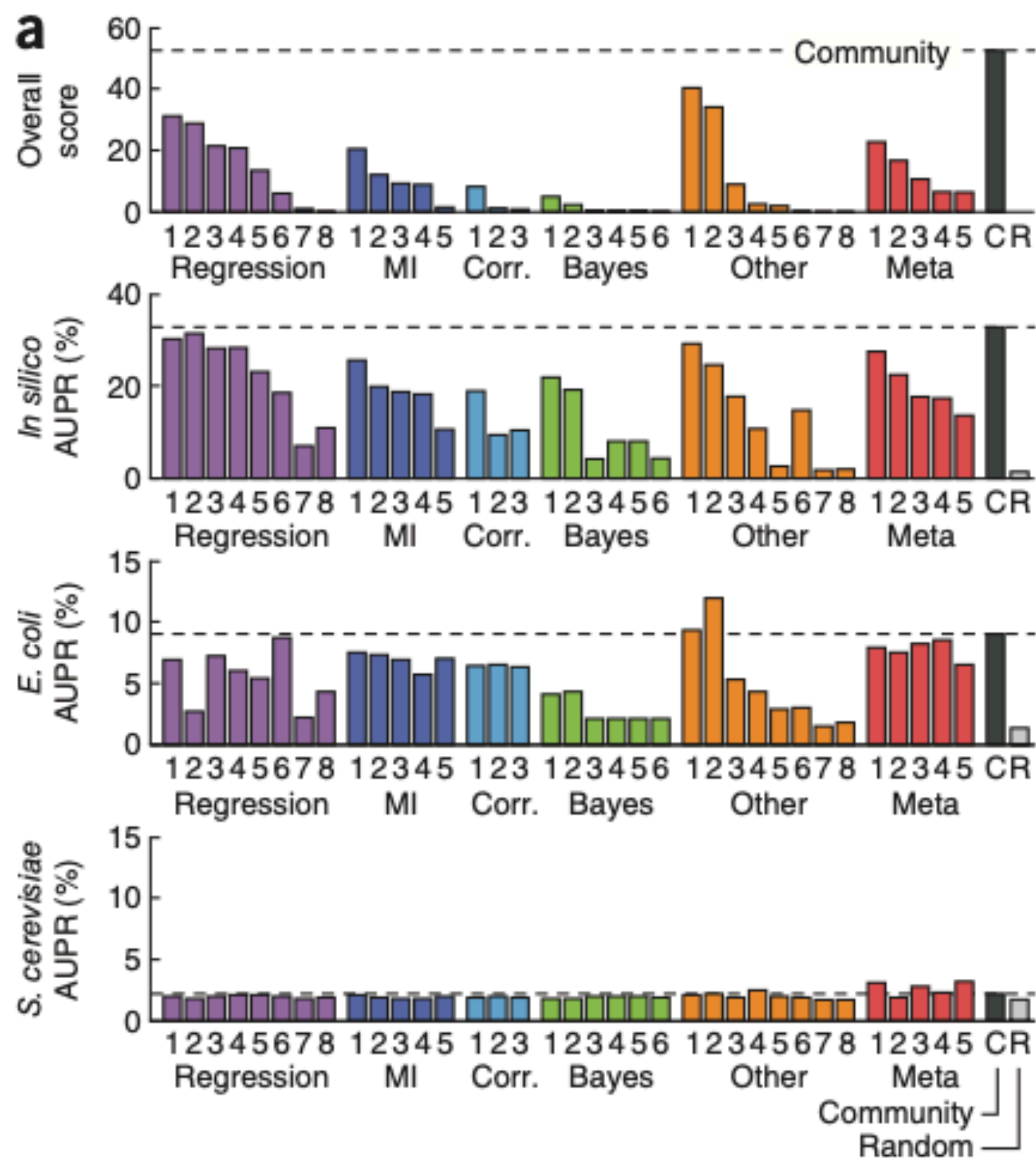
1	Simulated annealing (catnet R package, http://cran.r-project.org/web/packages/catnet/), aggregation of three runs.	—
2	Simulated annealing (catnet R package, hyperlink above).	—
3	Max-min parent and children algorithm (MMPC), bootstrapped data sets.	40
4	Markov blanket algorithm (HITON-PC), bootstrapped data sets.	41
5	Markov boundary induction algorithm (TIE*), bootstrapped data sets.	42
6	Models transcription factor perturbation data and time series using dynamic Bayesian networks (Infer.NET toolbox, http://research.microsoft.com/infernet/).	— ^a

Other approaches: network inference by heterogeneous and novel methods.

1	GENIE3: a Random Forest is trained to predict target gene expression. Putative transcription factors are selected as tree nodes if they consistently reduce the variance of the target.	19 ^a
2	Codependencies between transcription factors and target genes are detected by the nonlinear correlation coefficient η^2 (two-way ANOVA). Transcription-factor perturbation data are up-weighted.	20 ^a
3	Transcription factors are selected by maximizing the conditional entropy for target genes, which are represented as Boolean vectors with probabilities to avoid discretization.	43 ^a
4	Transcription factors are preselected from transcription-factor perturbation data or by Pearson's correlation and then tested by iterative Bayesian model averaging (BMA).	44
5	A Gaussian noise model is used to estimate whether the expression of a target gene changes in transcription-factor perturbation measurements.	45
6	After scaling, target genes are clustered by Pearson's correlation. A neural network is trained (genetic algorithm) and parameterized (back-propagation).	46 ^a
7	Data is discretized by Gaussian mixture models and clustering; interactions are detected by generalized logical network modeling (χ^2 test).	47 ^a
8	The χ^2 test is applied to evaluate the probability of a shift in transcription-factor and target-gene expression in transcription-factor perturbation experiments.	47 ^a

Meta predictors: (i) apply multiple inference approaches and (ii) compute aggregate scores.

1	(i) z scores for target genes in transcription-factor knockout data, time-lagged CLR for time series, and linear ordinary differential-equation models constrained by Lasso (Inferelator); (ii) resampling approach.	48 ^a
2	(i) Pearson's correlation, mutual information and CLR; (ii) rank average.	—
3	(i) Calculates target-gene responses in transcription-factor knockout data, applies full-order, partial correlation and transcription factor-target codeviation analysis; (ii) weighted average with weights trained on simulated data.	— ^a
4	(i) CLR filtered by negative Pearson's correlation, least-angle regression (LARS) of time series, and transcription factor perturbation data; (2) combination by z scores.	49
5	(i) Pearson's correlation, differential expression (limma), and time-series analysis (maSigPro); (ii) naive Bayes.	— ^a



Pearson correlation coefficient

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$$

where

- cov is the [covariance](#)
- σ_X is the [standard deviation](#) of X
- σ_Y is the standard deviation of Y .

$$r = \frac{n(\sum xy) - (\sum x)(\sum y)}{\sqrt{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}}$$

Mutual information

Let (X, Y) be a pair of random variables with values over the space $\mathcal{X} \times \mathcal{Y}$. If their joint distribution is $P_{(X,Y)}$ and the marginal distributions are P_X and P_Y , the mutual information is defined as

$$I(X; Y) = D_{\text{KL}}(P_{(X,Y)} \| P_X \otimes P_Y)$$

where D_{KL} is the [Kullback–Leibler divergence](#), and $P_X \otimes P_Y$ is the [outer product](#) distribution which assigns probability $P_X(x) \cdot P_Y(y)$ to each (x, y) .

Notice, as per property of the [Kullback–Leibler divergence](#), that $I(X; Y)$ is equal to zero precisely when the joint distribution coincides with the product of the marginals, i.e. when X and Y are independent (and hence observing Y tells you nothing about X). $I(X; Y)$ is non-negative, it is a measure of the price for encoding (X, Y) as a pair of independent random variables when in reality they are not.

If the [natural logarithm](#) is used, the unit of mutual information is the [nat](#). If the [log base 2](#) is used, the unit of mutual information is the [shannon](#), also known as the bit. If the [log base 10](#) is used, the unit of mutual information is the [hartley](#), also known as the ban or the dit.

Mutual information

In terms of PMFs for discrete distributions [\[edit \]](#)

The mutual information of two jointly discrete random variables X and Y is calculated as a double sum:[\[3\]:20](#)

$$I(X; Y) = \sum_{y \in \mathcal{Y}} \sum_{x \in \mathcal{X}} P_{(X,Y)}(x, y) \log \left(\frac{P_{(X,Y)}(x, y)}{P_X(x) P_Y(y)} \right), \quad (\text{Eq.1})$$

$$I(X; Y) = \sum_{ij} P(x_i, y_j) \log \frac{p(x_i, y_j)}{p(x_i)p(y_j)}$$

where $P_{(X,Y)}$ is the [joint probability mass function](#) of X and Y , and P_X and P_Y are the [marginal probability](#) mass functions of X and Y respectively.

In terms of PDFs for continuous distributions [\[edit \]](#)

In the case of jointly continuous random variables, the double sum is replaced by a [double integral](#):[\[3\]:251](#)

$$I(X; Y) = \int_{\mathcal{Y}} \int_{\mathcal{X}} P_{(X,Y)}(x, y) \log \left(\frac{P_{(X,Y)}(x, y)}{P_X(x) P_Y(y)} \right) dx dy, \quad (\text{Eq.2})$$

where $P_{(X,Y)}$ is now the joint probability *density* function of X and Y , and P_X and P_Y are the marginal probability density functions of X and Y respectively.

Mutual information can be equivalently expressed as:

$$\begin{aligned} I(X; Y) &\equiv H(X) - H(X | Y) \\ &\equiv H(Y) - H(Y | X) \\ &\equiv H(X) + H(Y) - H(X, Y) \\ &\equiv H(X, Y) - H(X | Y) - H(Y | X) \end{aligned}$$

where $H(X)$ and $H(Y)$ are the marginal [entropies](#), $H(X | Y)$ and $H(Y | X)$ are the [conditional entropies](#), and $H(X, Y)$ is the [joint entropy](#) of X and Y .

Regression-based method

$$X_g = f_g(X_{\mathcal{T}_g}) + \varepsilon ,$$

$X_{\mathcal{T}_g} = \{X_t , t \in \mathcal{T}_g\}$ is the set of expression levels of the candidate transcription factors for gene g , and ε is some noise

$$f_g(X_{\mathcal{T}_g}) = \sum_{t \in \mathcal{T}_g} \beta_{t,g} X_t ,$$

Any linear or nonlinear statistical method for regression can potentially be used to infer f_g from the observed expression data.

*TIGRESS: Trustful Inference of Gene Regulation using Stability Selection

GRN inference with feature selection methods

Many popular methods for GRN inference are based on such a score. For example, the correlation or mutual information between the expression levels of t and g along the different experiments is a popular way to score candidate regulations [12-14]. A drawback of such direct approaches is that it is then difficult to separate direct from indirect regulations. For example, if t_1 regulates t_2 which itself regulates g , then the correlation or mutual information between t_1 and g is likely to be large, although (t_1, g) is not

a direct regulation. Similarly, if t_1 regulates both t_2 and g , then t_2 and g will probably be very correlated, even if there is no direct regulation between them. In order to overcome this problem, a possible strategy is to post-process the predicted regulations and try to remove regulations likely to be indirect because they are already explained by other regulations [13]. Another strategy is, given a target gene $g \in \mathcal{G}$, to *jointly* estimate the scores $s(t, g)$ for all candidate regulators $t \in \mathcal{T}_g$ simultaneously, with a method able to capture the fact that a large score for a candidate regulation (t, g) is not needed if the apparent correlation between t and g is already explained by other, more likely regulations.

Mathematically, the latter strategy is closely related to the problem of *feature selection* in statistics, as already observed and exploited by several authors [20,23]. More specifically, for each target gene $g \in \mathcal{G}$, we consider the regression problem where we wish to predict the expression level of g from the expression level of its candidate regulators $t \in \mathcal{T}_g$:

$$X_g = f_g(X_{\mathcal{T}_g}) + \epsilon, \quad (1)$$

where X_i represents the expression level of the i -th gene across different experiments (modeled as a random variable), $X_{\mathcal{T}_g} = \{X_t, t \in \mathcal{T}_g\}$ is the set of expression levels of the candidate transcription factors for gene g , and ϵ is some noise. Any linear or nonlinear statistical method for regression can potentially be used to infer f_g from the observed expression data. However, we are not directly interested in the regression function f_g , but instead in the identification of a small set of transcription factors which are sufficient to provide a good model for X_g . We therefore need a score $s_g(t)$ for each candidate transcription factor $t \in \mathcal{T}_g$ to assess how likely it is to be involved in the regression model f_g . For example, if we model f_g as a linear function

$$f_g(X_{\mathcal{T}_g}) = \sum_{t \in \mathcal{T}_g} \beta_{t,g} X_t, \quad (2)$$

then the score $s_g(t)$ should typically assess the probability that $\beta_{t,g}$ is non-zero [23]. More general models are possible, for example [20] model f_g with a random forest [29] and score a predictor $s_g(t)$ with a variable importance measure specific to this model. Once a score $s_g(t)$ is chosen to assess the significance of each transcription factor in the target-gene-specific regression model (1), we can combine them across all target genes by defining the score of a candidate regulation $(t, g) \in \mathcal{E}$ as $s(t, g) = s_g(t)$, and rank all candidate regulations by decreasing score for GRN inference.

✱

Feature selection with LARS and stability selection

We now propose a new scoring function $s_g(t)$ to assess the significance of a transcription factor $t \in \mathcal{T}_g$ in the regression model (1). Our starting point to define the scoring function is the LARS method for feature selection in regression [24]. LARS models the regression function (1) linearly, *i.e.* it models the expression of a target gene as a linear combination of the expression of its transcription factors, as in (2). Starting from a constant model where no TF is used, it iteratively adds TF in the model to refine the prediction of X_g . Contrary to classical forward stepwise feature selection [30,31], LARS does not fully re-optimize the fitted model when a new TF is added to the model, but only refines it partially. This results in a statistically sound procedure for feature selection, akin to forward stage-wise linear regression and the Lasso [27,31], and a very efficient computational procedure. In practice, after L steps of the LARS iteration, we obtain a ranked list of L TF selected for their ability to predict the expression of the target gene of interest. Efficient implementations of LARS exist in various programming languages including R (`lars` package, [24]) and MATLAB (SPAMS toolbox, [32]). Since the selection of TF is iterative, LARS has the potential to disregard indirect regulations.

The direct use of LARS to score candidate regulations has, however, two shortcomings. First, LARS can be very sensitive and unstable in terms of selected features when there exist high correlations between different explanatory variables. Second, it only provides a ranking of the TF, for each TG of interest, but does not provide a score $s_g(t)$ to quantify the evidence that a TF t regulates a target gene g . Since we want to aggregate the predicted regulations across all target genes to obtain a global ranking of all candidate regulations, we need such a score.

least angle regression (LARS)

To overcome both issues, we do not directly score candidate regulations with the LARS, but instead perform a procedure known as *stability selection* [26] on top of LARS. The general idea of stability selection is to run a feature selection method many times on randomly perturbed data, and score each feature by the number of times it was selected. It was shown that stability selection can reduce the sensitivity of LARS and Lasso to correlated features, and improve their ability to select correct features [25,26]. In addition, it provides a score for each feature, which can then be aggregated over different regression problems, *i.e.* different target genes in our case. More precisely, to score the candidate target genes $t \in \mathcal{T}_g$ of a given target gene g using LARS with stability selection, we fix a (large) number of iterations R , and repeat $R/2$ times the following iterations: we randomly split the experiments into two halves of equal or approximately equal size, we multiply the expression levels of the candidate transcription factors in $\mathcal{T}_g$ on each microarray by a random number uniformly sampled on the interval $[\alpha, 1]$ for some $0 \leq \alpha \leq 1$, and we run the LARS method for $L > 0$ steps on the two resulting reduced and reweighed expression matrices. We therefore perform a total of R LARS runs on randomly modified expression matrices. For each run, the result of LARS after L steps is a ranked list of L TF. After the R runs, we record for each $g \in \mathcal{G}$, $t \in \mathcal{T}_g$ and $l \in [1, L]$ the frequency $F(g, t, l)$ with which the TF t was selected by the LARS in the top l features to predict the expression of gene g . We thus obtain a final score between 0 and 1, 1 meaning that t is always selected by LARS in the top l features to predict the expression level of g , and 0 that it is never selected. Figure 1 displays graphically these frequencies, for a given gene g fixed, all candidate TF in $\mathcal{T}_g$, and $l = 1, \dots, 15$. When l increases, the frequency $F(g, t, l)$ for fixed g and t is non-decreasing because the LARS method selects increasing sets of TF at each step. In addition, since the total number of TF selected after l LARS steps is always equal to l , taking the average over the R LARS runs leads to the equality $\sum_{t \in \mathcal{T}_g} F(g, t, l) = l$, for any gene g and LARS step l .

GENIE3: Inferring Regulatory Networks from Expression Data Using Tree-Based Methods

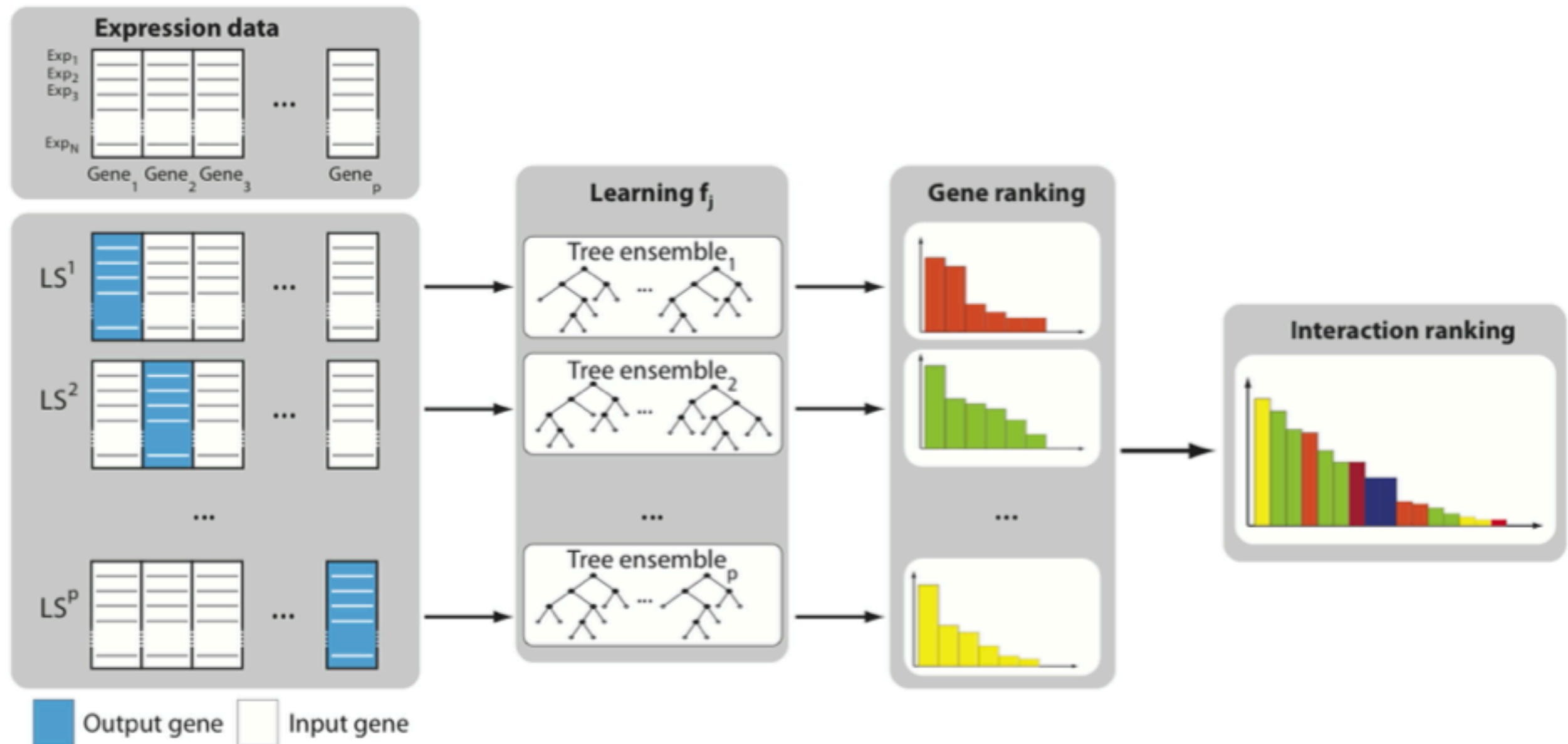


Figure 1. GENIE3 procedure. For each gene $j=1, \dots, p$, a learning sample LS^j is generated with expression levels of j as output values and expression levels of all other genes as input values. A function f_j is learned from LS^j and a local ranking of all genes except j is computed. The p local rankings are then aggregated to get a global ranking of all regulatory links.

doi:10.1371/journal.pone.0012776.g001

* Network Inference with Tree-based Methods

The basic idea of our procedure is to decompose the problem of recovering a network involving p genes into p different subproblems, where each of these subproblems consists in identifying the regulators of one of the genes of the network. Exploiting expression data, the identification of the regulatory genes for a given target gene is defined as determining the subset of genes whose expression directly influences or is predictive of the expression of the target gene. Within the framework of supervised learning, this problem is equivalent to a feature selection problem. In this context, our solution will exploit the embedded feature ranking mechanism of tree-based ensemble methods.

We first describe our procedure to solve the network inference problem using feature selection techniques and then specialize it to the case of tree-based ensemble methods.

Network Inference as a Feature Selection Problem

Our method makes the assumption that the expression of each gene in a given condition is a function of the expression of the other genes in the network (plus some random noise). Denoting by $\mathbf{x}_k^{-j}$ the vector containing the expression values in the k th experiment of all genes except gene j :

we assume that we can write:

$$x_k^j = f_j(\mathbf{x}_k^{-j}) + \varepsilon_k, \forall k \quad (1)$$

where ε_k is a random noise with zero mean (conditionally to $\mathbf{x}_k^{-j}$). We further make the assumption that the function f_j only exploits the expression in $\mathbf{x}^{-j}$ of the genes that are direct regulators of gene j , i.e. genes that are directly connected to gene j in the targeted network. Recovering the regulatory links pointing to gene j thus amounts at finding those genes whose expression is predictive of the expression of the target gene. In machine learning terminology, this can be considered as a feature selection problem (in regression) for which many solutions exist [28]. We assume here the use of a feature ranking technique that, instead of directly returning a feature subset, yields a ranking of the features from the most relevant to the less relevant for predicting the output.

The proposed network inference procedure is illustrated in Figure 1 and works as follows:

- For $j = 1$ to p :
 - Generate the learning sample of input-output pairs for gene j :

$$LS^j = \{(\mathbf{x}_k^{-j}, x_k^j), k = 1, \dots, N\}.$$

- Use a feature selection technique on LS^j to compute confidence levels $w_{ij}, \forall i \neq j$, for all genes except gene j itself.
- Aggregate the p individual gene rankings to get a global ranking of all regulatory links.

Note that depending of the interpretation of the weights w_{ij} , their aggregation to get a global ranking of regulatory links is not trivial. We will see in the context of tree-based methods that it requires to normalize each expression vector appropriately.

Gene Ranking with Tree-based Methods

The nature of the problem and the proposed solution put some constraints on candidate feature selection techniques. The nature of the functions f_j is unknown but they are expected to involve the expression of several genes (combinatorial regulation) and to be non-linear. The number of input features in each of these problems is typically much greater than the number of observations. Computationally, since the identification of a network involving p genes requires to rerun the algorithm p times, it is also of interest for this algorithm to be fast and to require as few manual tuning as possible. Tree-based ensemble methods are good candidates for that purpose. These methods do not make any assumption about the nature of the target function, can potentially deal with interacting features and non-linearity. They work well in the presence of a large number of features, are fast to compute, scalable, and essentially parameter-free (see [29] for a review).

We first briefly describe these methods and their built-in feature ranking mechanism and then discuss their use in the context of the network inference procedure described in the previous section.

*

Tree-based Ensemble Methods

Each subproblem, defined by a learning sample LS^j , is a supervised (non-parametric) regression problem. Using square error loss, each problem amounts at finding a function f_j that minimizes the following error:

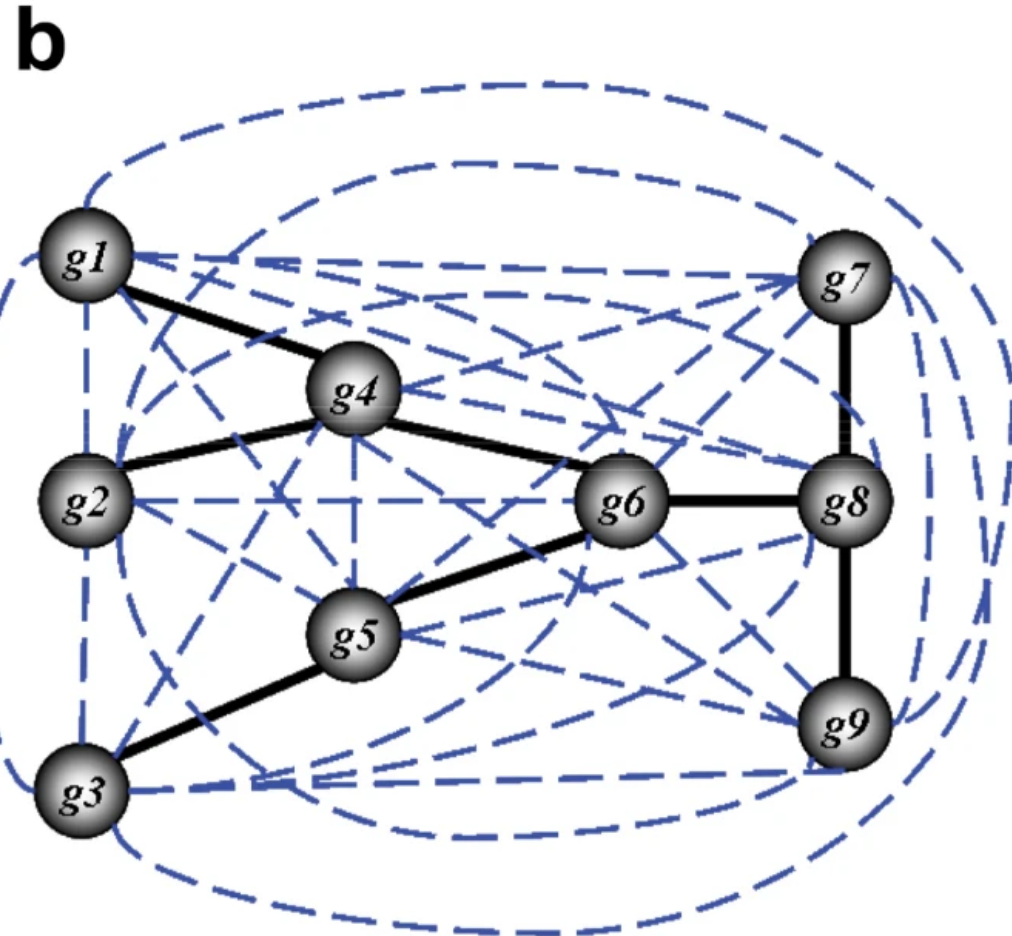
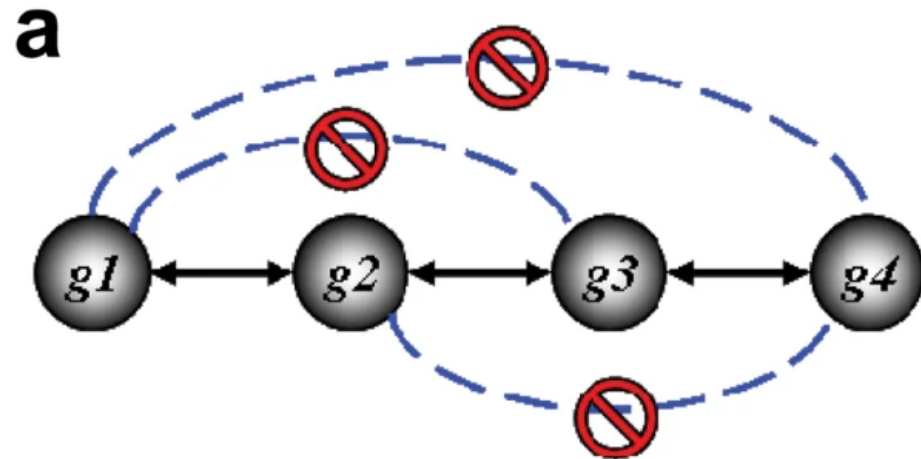
$$\sum_{k=1}^N (x_k^j - f_j(\mathbf{x}_k^{-j}))^2. \quad (2)$$

Regression trees [30] solve this problem by developing tree structured models. The basic idea of this method is to recursively split the learning sample with binary tests based each on one input variable (selected in $\mathbf{x}^{-j}$), trying to reduce as much as possible the variance of the output variable (x^j) in the resulting subsets of samples. Candidate splits for numerical variables typically compare the input variable values with a threshold which is determined during the tree growing.

Single trees are usually very much improved by ensemble methods, which average the predictions of several trees. In our network inference procedure, we compare two tree-based ensemble methods based on randomization, namely Random Forests [31] and Extra-Trees [32]. In a Random Forests ensemble, each tree is built on a bootstrap sample from the original learning sample and, at each test node, K attributes are selected at random among all candidate attributes before determining the best split. In the Extra-Trees method on the other hand, each tree is built from the original learning sample and at each test node, the best split is determined among K random splits, each determined by randomly selecting one input (without replacement) and a threshold. For these two methods, we will grow ensembles of 1000 trees and we will consider two values of the main parameter of these methods: $K = \sqrt{p-1}$ and $K = p-1$, where $p-1$ is the number of inputs, equal to the number of potential regulators of each gene. Empirical validations in [32] have shown that these two values of K were near-optimal in terms of predictive accuracy on several problems. Note however that we do not exclude that better results could be obtained with other settings of K .

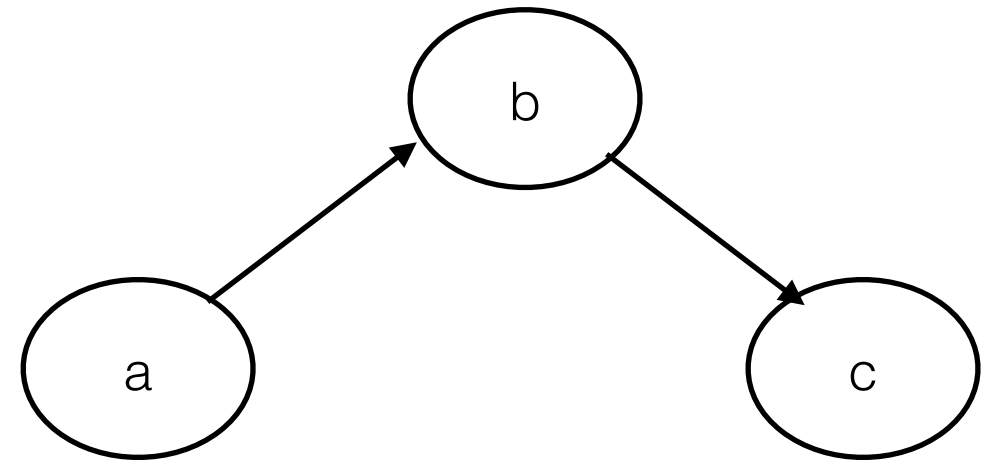
ARACNE: An Algorithm for the Reconstruction of Gene Regulatory Networks in a Mammalian Cellular Context

Data Processing Inequality:

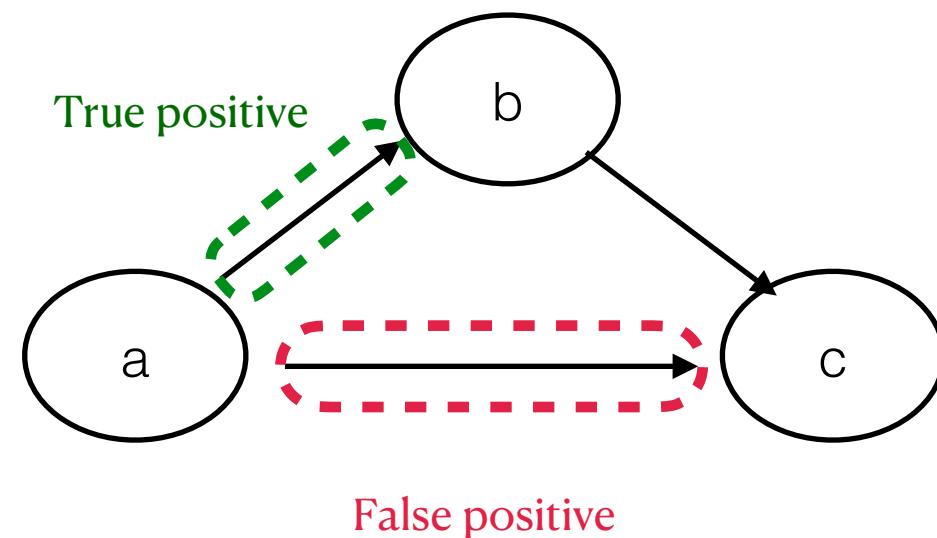


Transitive interaction:

Gold standard



Predicted



Context Likelihood of Relatedness

$$M^{\text{stat}} = \begin{pmatrix} I(x_1; x_1) & I(x_1; x_2) & \cdots & I(x_1; x_N) \\ I(x_2; x_1) & I(x_2; x_2) & \cdots & I(x_2; x_N) \\ \vdots & \vdots & \ddots & \vdots \\ I(x_N; x_1) & I(x_N; x_2) & \cdots & I(x_N; x_N) \end{pmatrix}.$$

Let M be a $N \times N$ matrix, with each entry, $M_{i,j}$, equals the pair-wise MI between a pair of variables, $I(x_i; x_j)$. In order to derive a CLR score for that pair of variables, $z(x_i, x_j)$, first compute a positive Z-score for $M_{i,j}$ with respect to the entries in the i 'th row of M , i.e.

$$z_i(x_i, x_j) = \max \left(0, \frac{M_{i,j} - \frac{\sum_j M_{i,j}}{N}}{\sigma_i} \right),$$

where σ_i is the standard deviation of the entries in the i 'th row of M . Second, compute a positive Z-score for $M_{i,j}$ with respect to the entries in the j 'th column of M , i.e.

$$z_j(x_i, x_j) = \max \left(0, \frac{M_{i,j} - \frac{\sum_i M_{i,j}}{N}}{\sigma_j} \right),$$

$$z(x_i, x_j) = \sqrt{z_i(x_i, x_j)^2 + z_j(x_i, x_j)^2}.$$

gene regulatory network inference from single cell

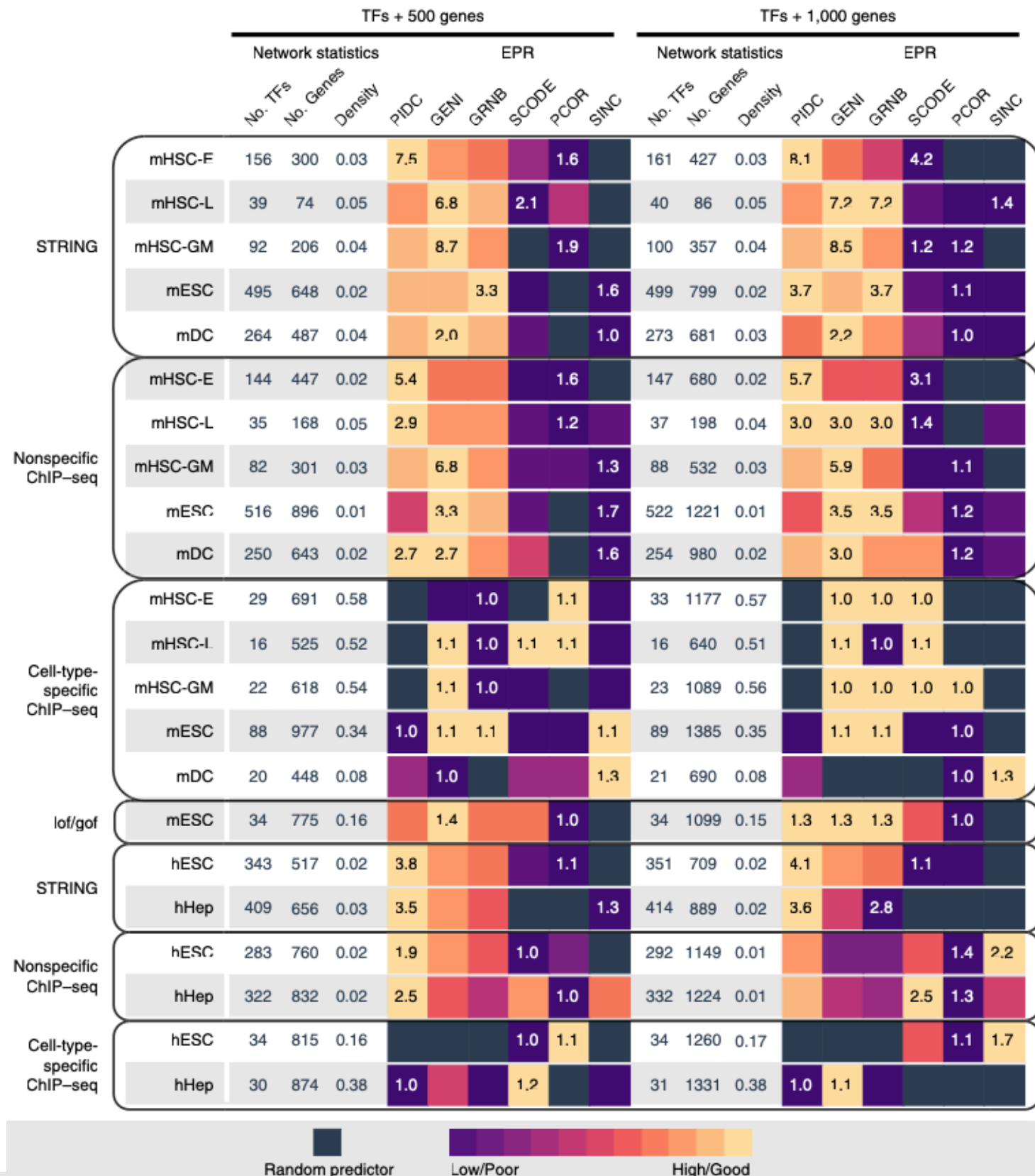
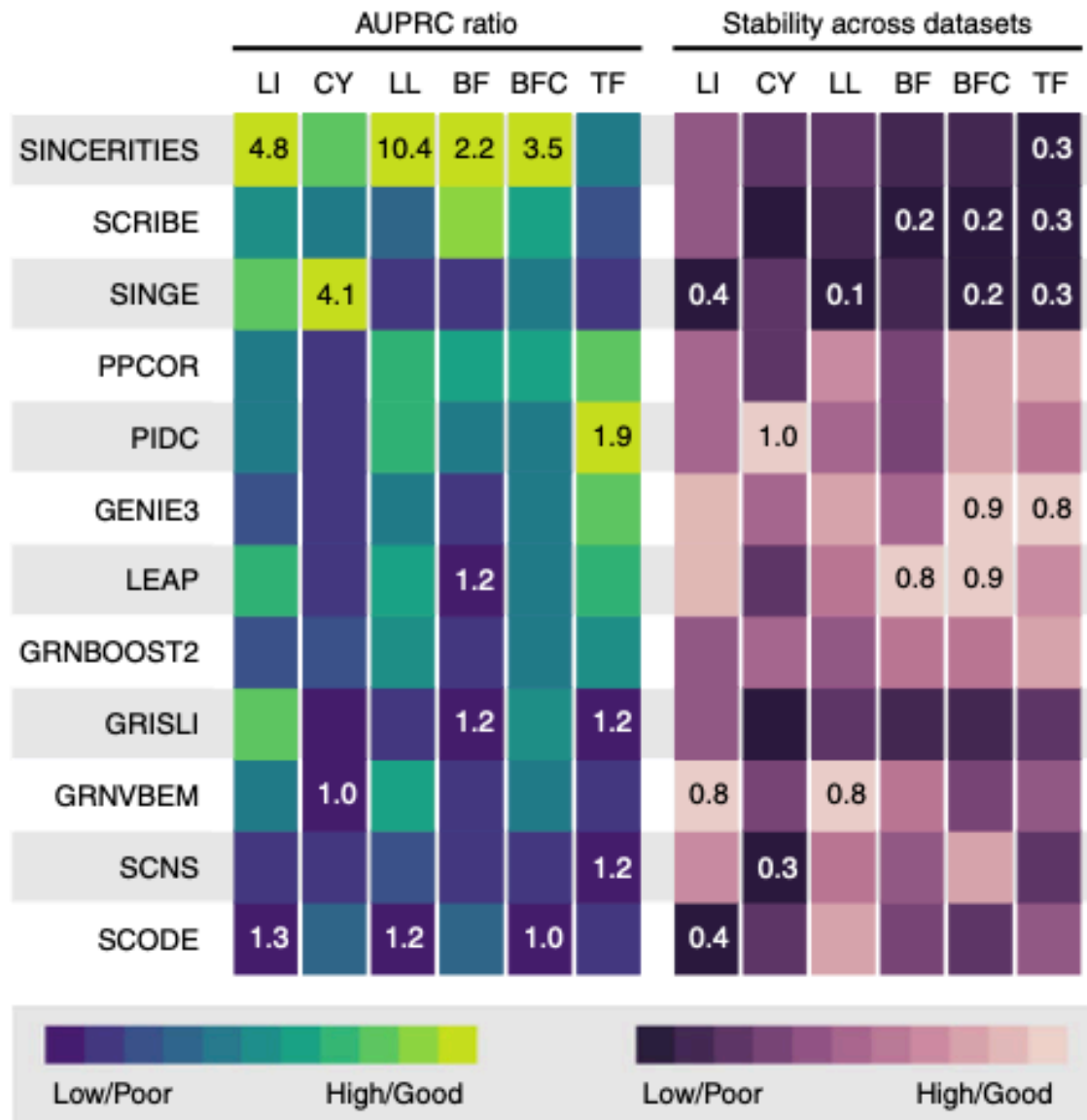
nature **methods**

ANALYSIS

<https://doi.org/10.1038/s41592-019-0690-6>

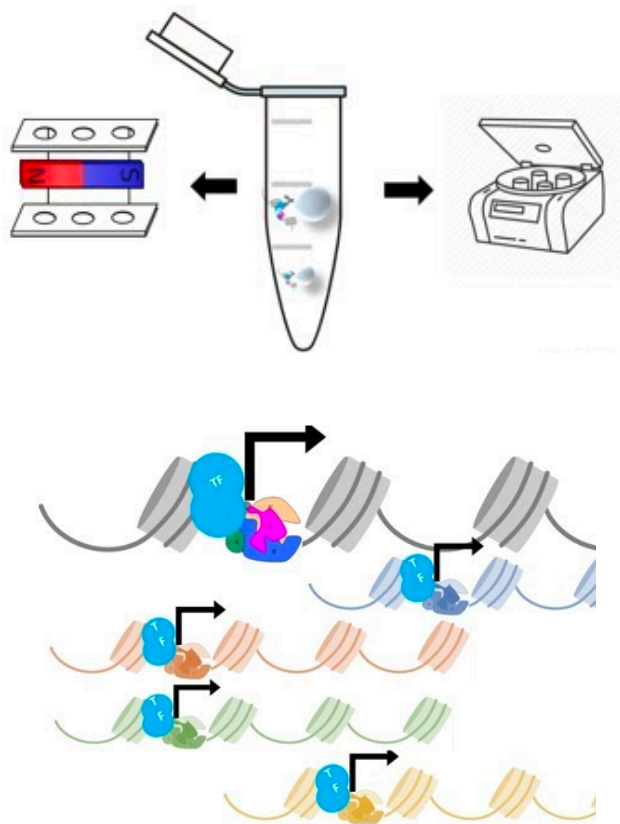
Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data

Aditya Pratapa¹, Amogh P. Jaliyal², Jeffrey N. Law², Aditya Bharadwaj¹ and T. M. Murali^{1*}



Unsupervised methods do not perform well

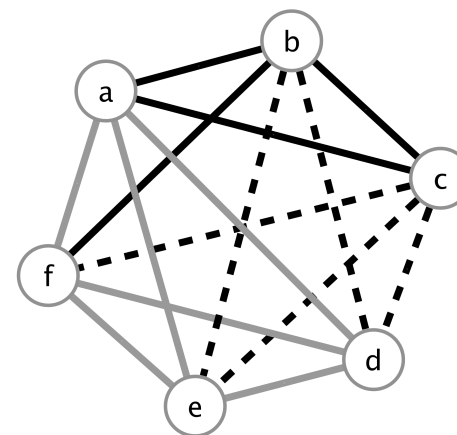
ChIP-Seq technology



ChIP-Seq
experiment
for TF b, c.



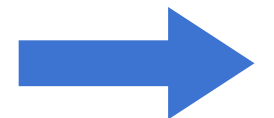
— unknown
— regulatory interaction
- - - no interaction



partially known GRN in the cell type

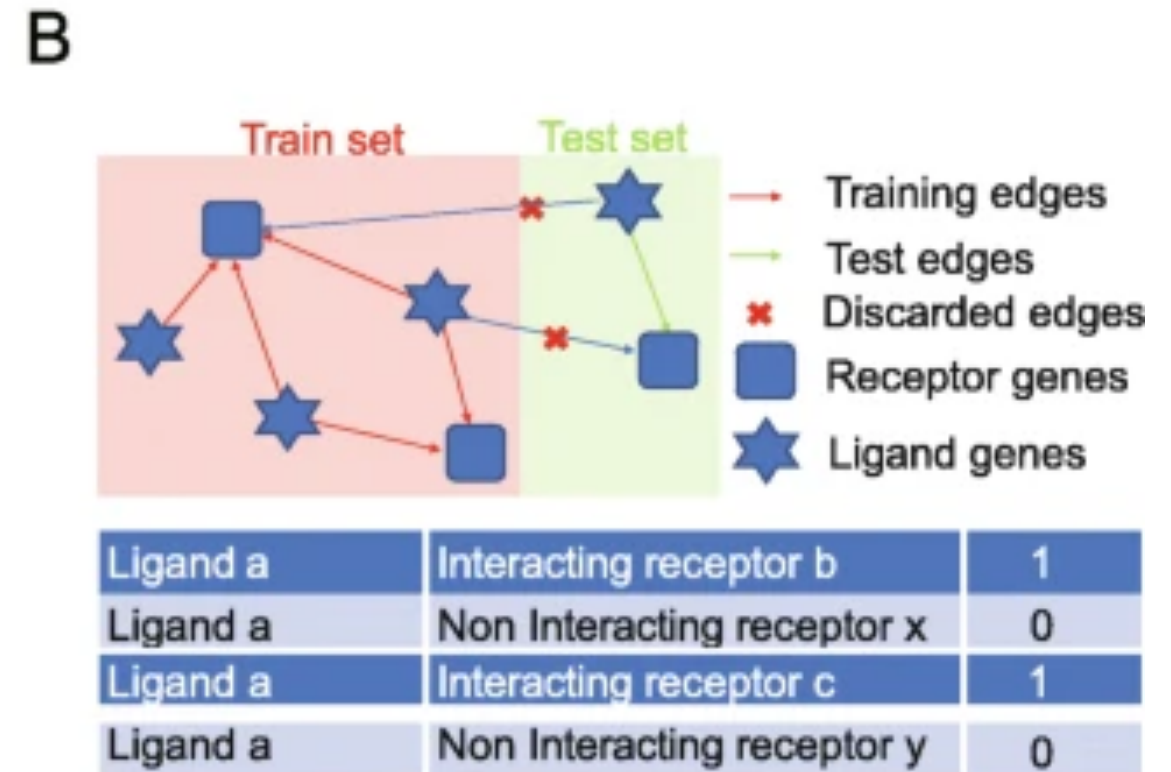
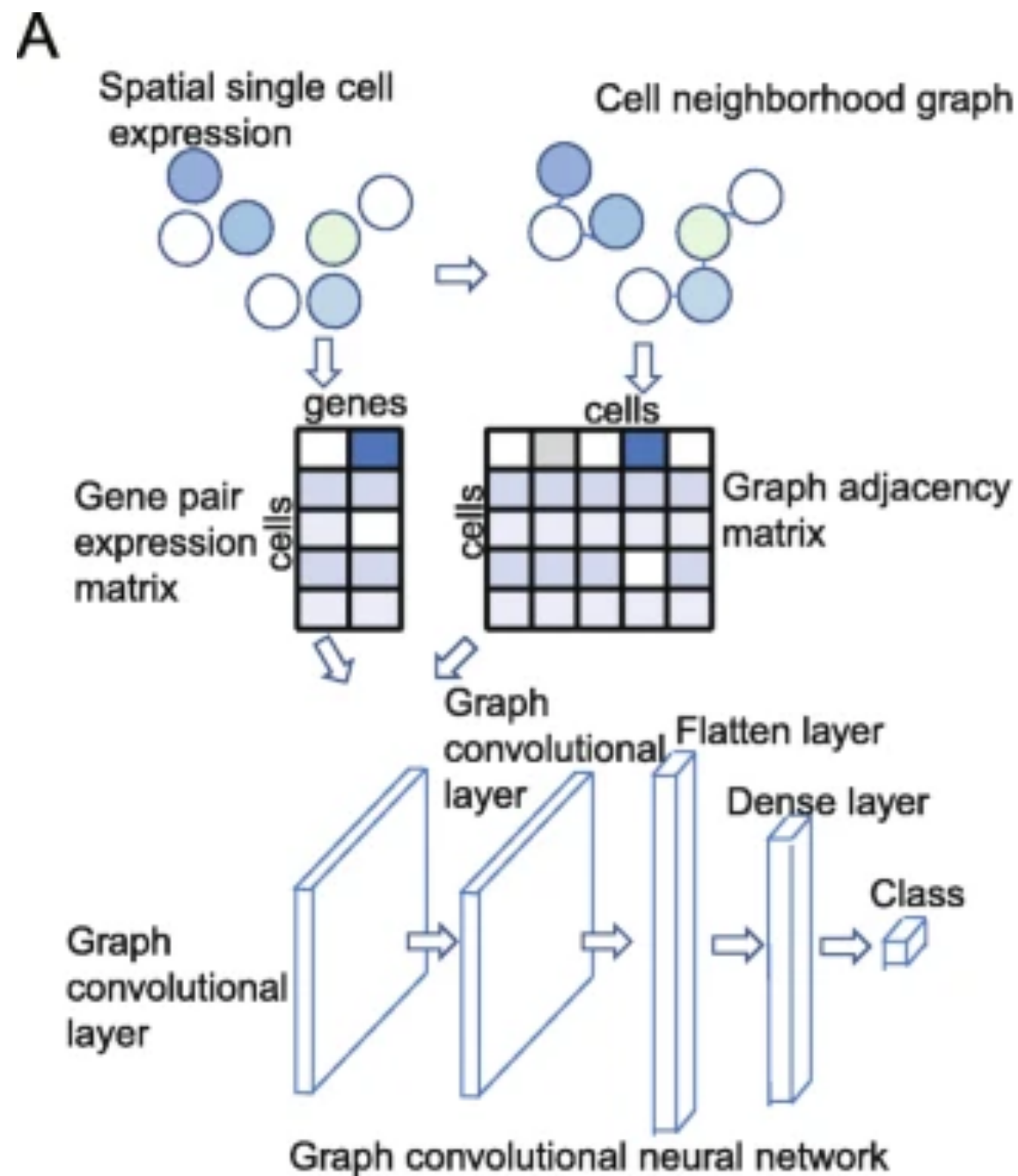
(b, a): 1
(b, c): 1
(b, d): 0
(b, e): 0
(b, f): 1
(c, a): 1
(c, b): 1
(c, d): 0
(c, e): 0
(c, f): 0

Used as
training data



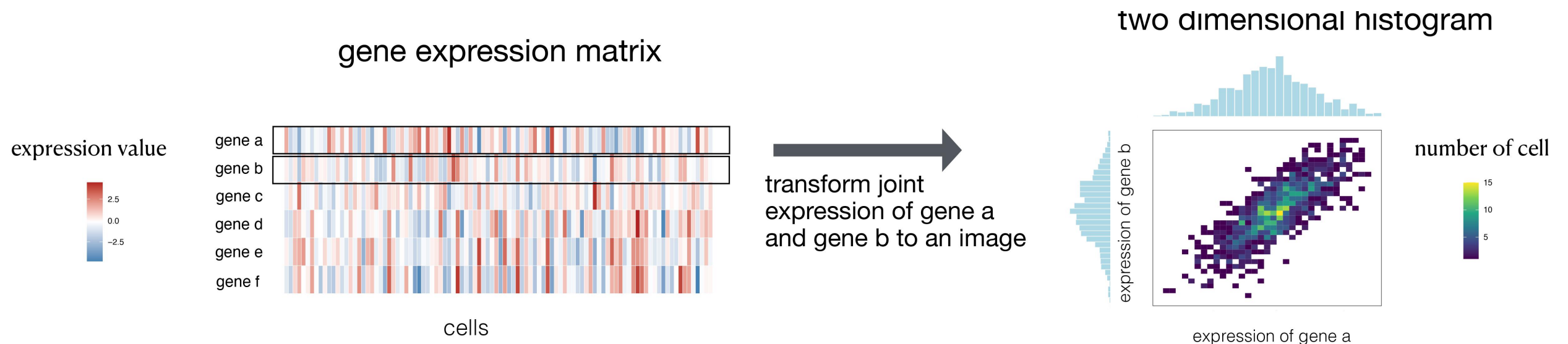
Supervised deep neural network is an alternative way to infer the GRN. It does not have any assumption on the distribution of gene expression.

GCNG: graph convolutional networks for inferring gene interaction from spatial transcriptomics data



DeepDRIM: A deep neural network to reconstruct GRN using single-cell RNA-seq data

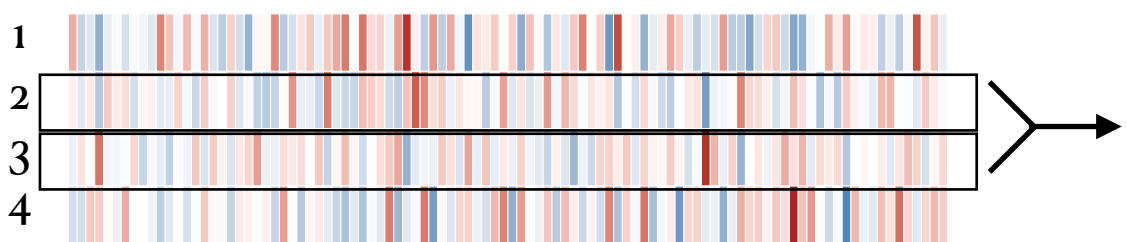
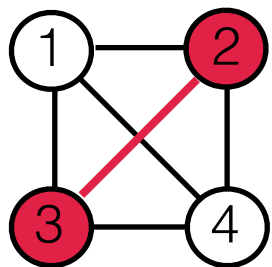
Only consider gene expression of the two genes may be Insufficient.



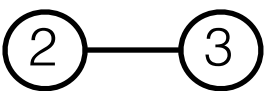
DeepDRIM: A deep neural network to reconstruct GRN using single-cell RNA-seq data

Primary image and neighbour images

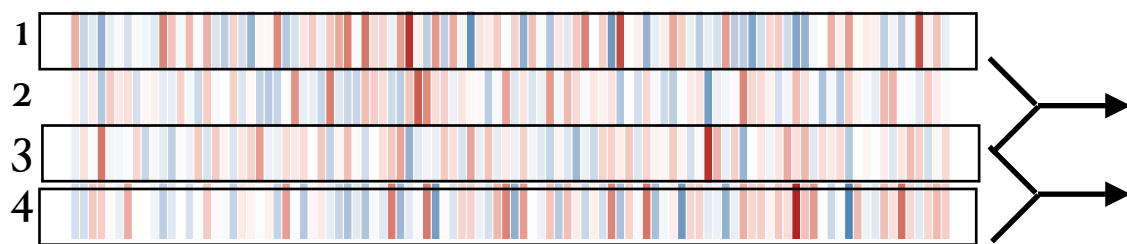
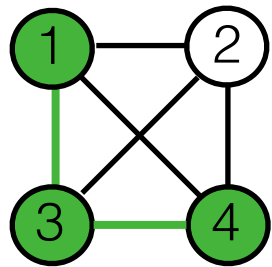
Target gene pair



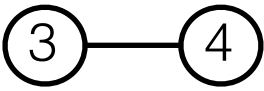
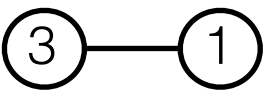
Primary image



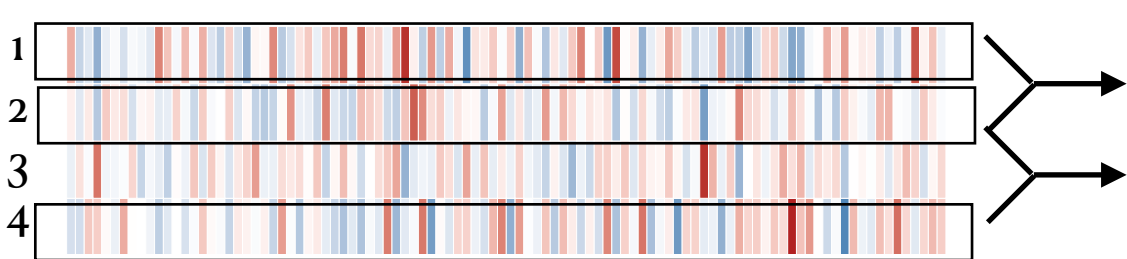
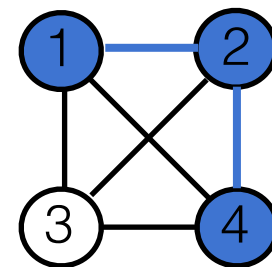
Neighbour of gene 3



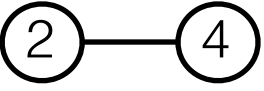
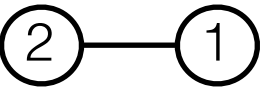
Neighbour images



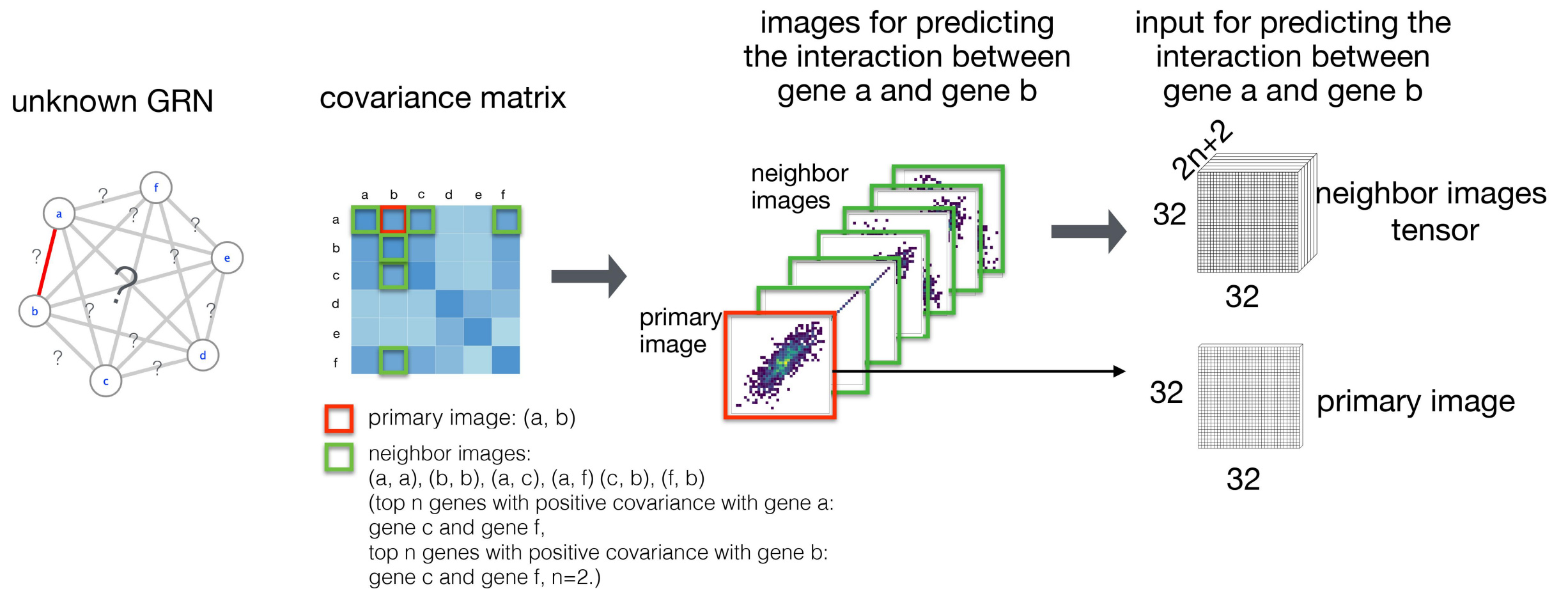
Neighbour of gene 2



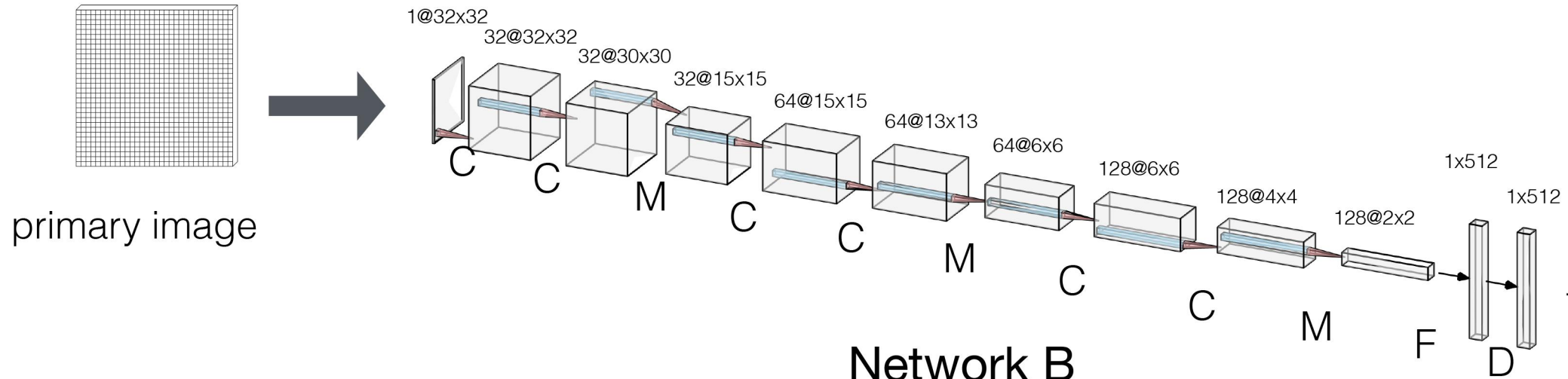
Neighbour images



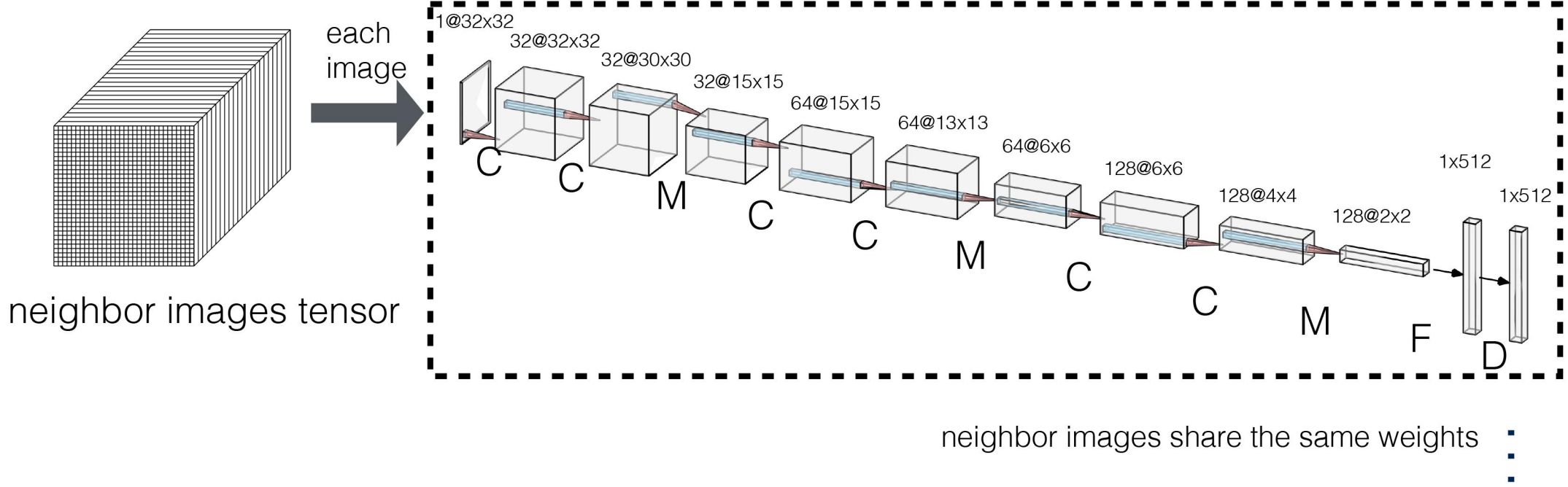
DeepDRIM: A deep neural network to reconstruct GRN using single-cell RNA-seq data



Network A



Network B



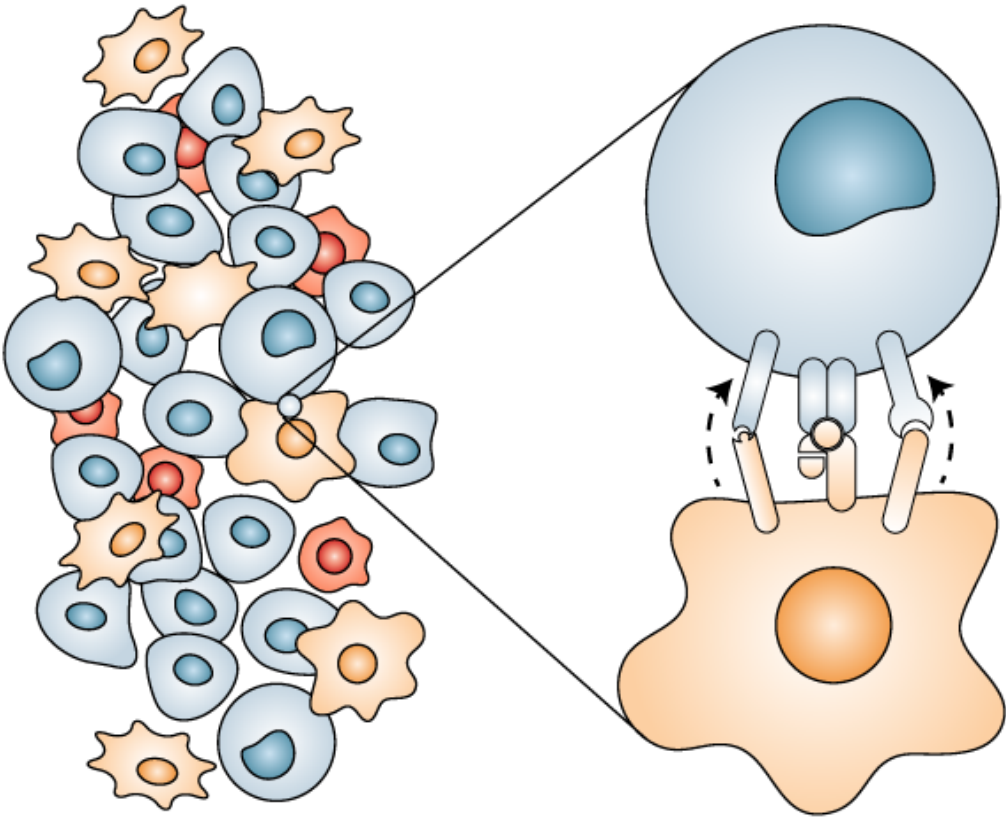
C: Convolution
M: Max-Pool
F: Flatten
D: Dense

concatenate

Cell Cell Interaction

- **Cellchat**
- **COMMOT**
- **FlowSig**
- **DeepLinc**
- **CellNest**

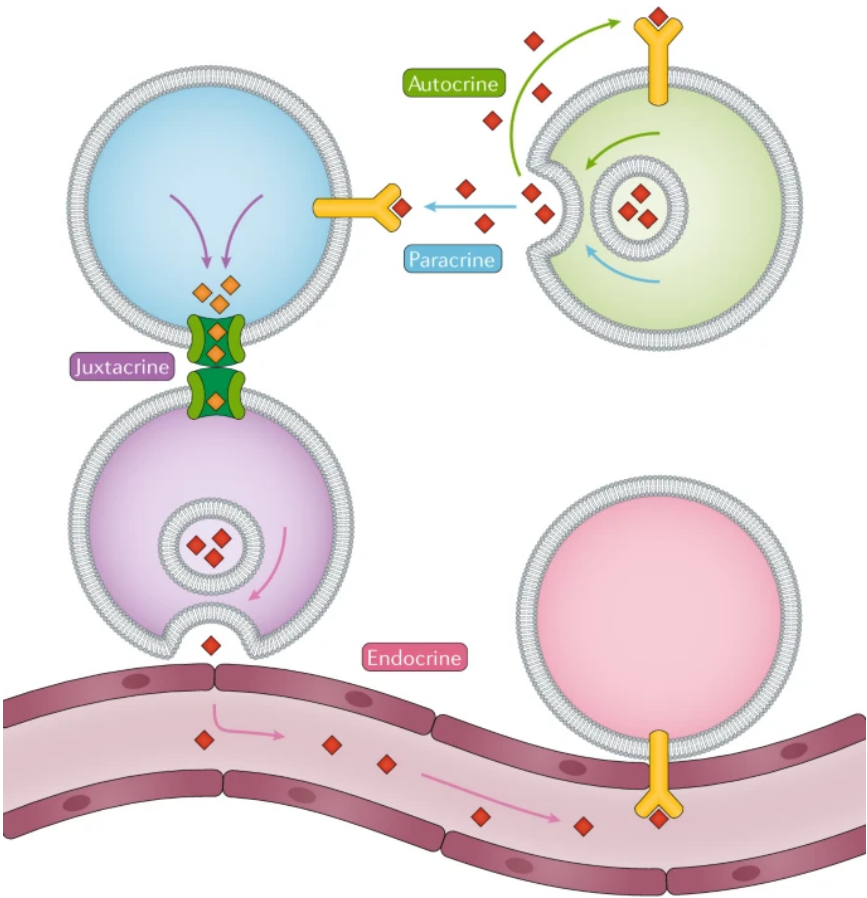
Cell Cell communication (CCI)



Ligand-receptor binding

Multicellular community

Cell-cell interaction

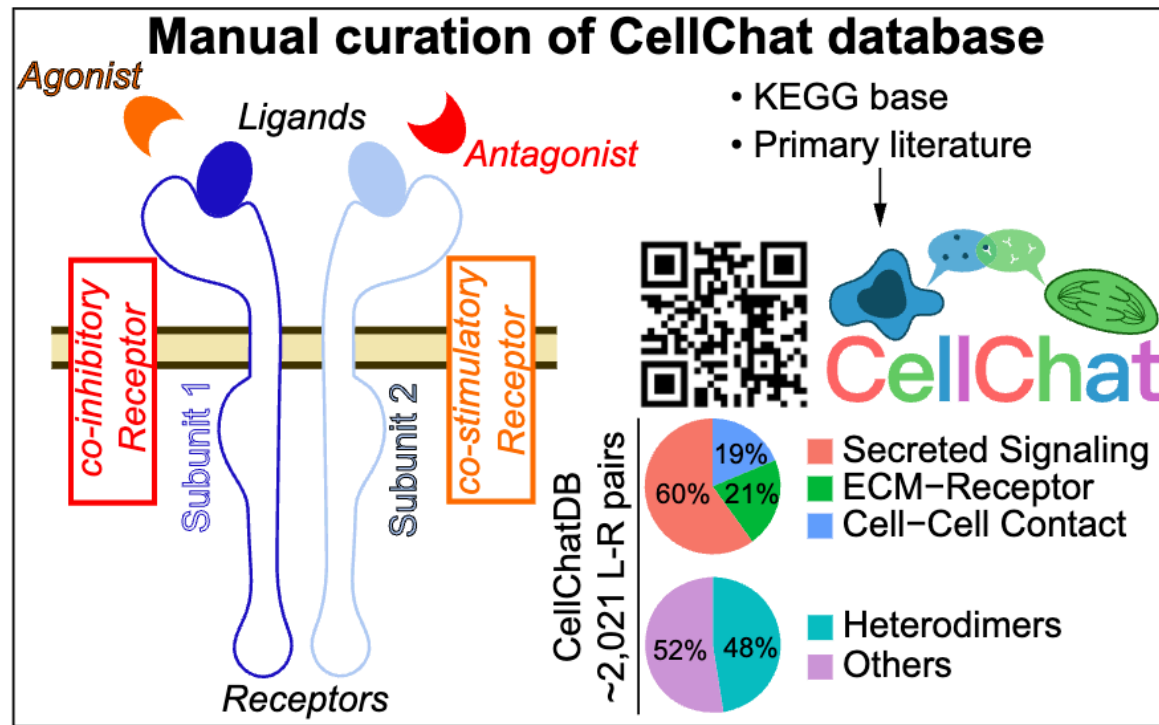


Various type of CCI

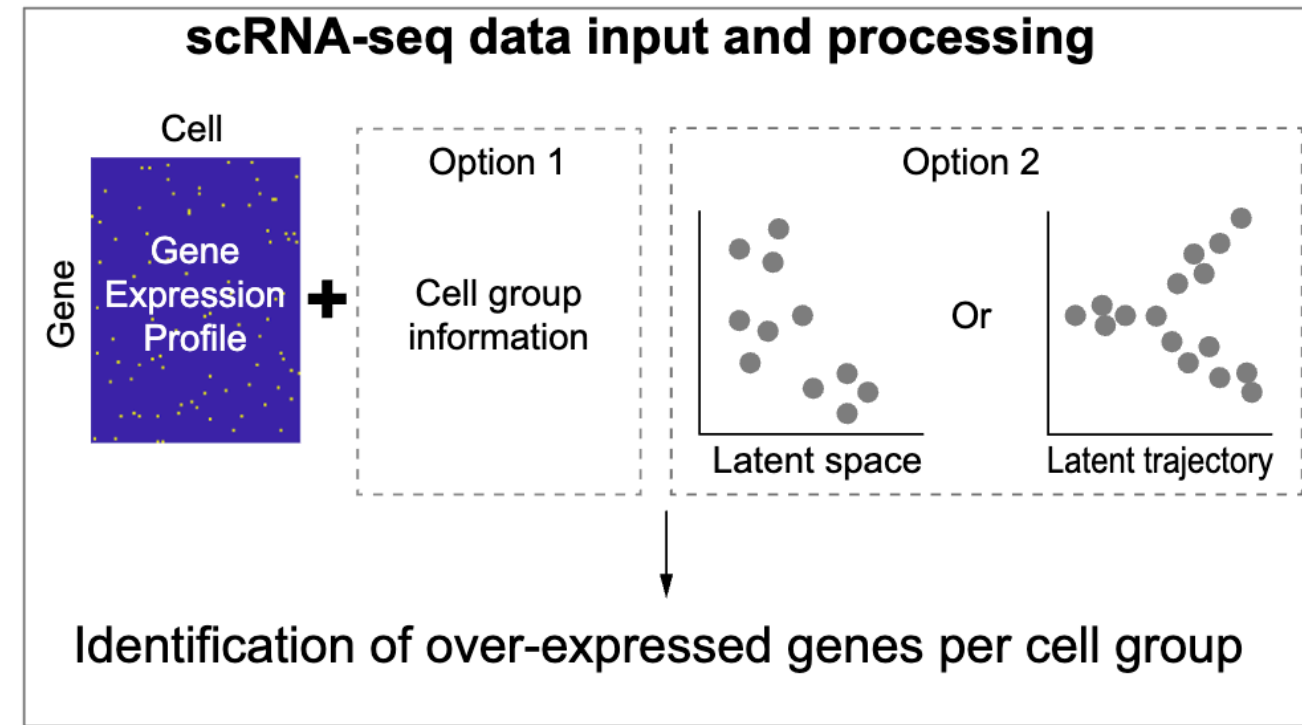
Cellchat

infer CCI from scRNA-seq data

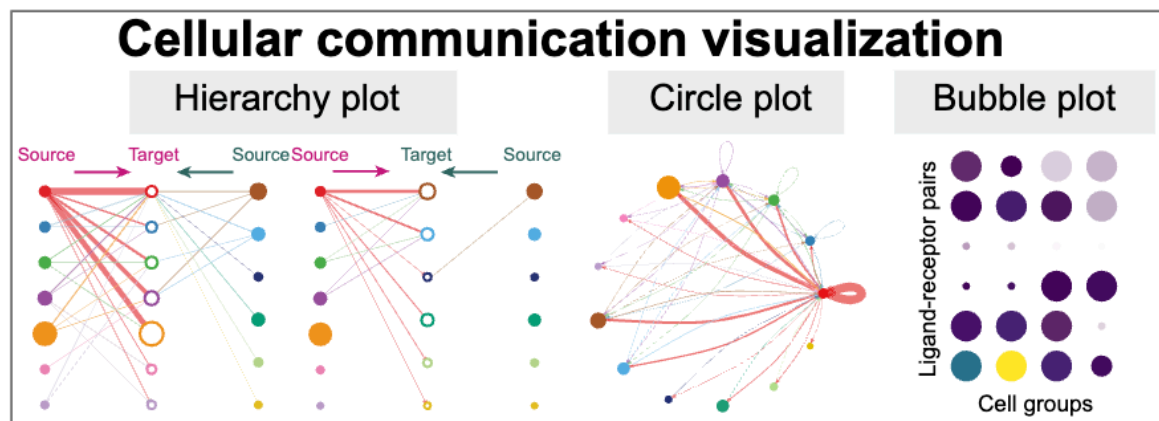
a



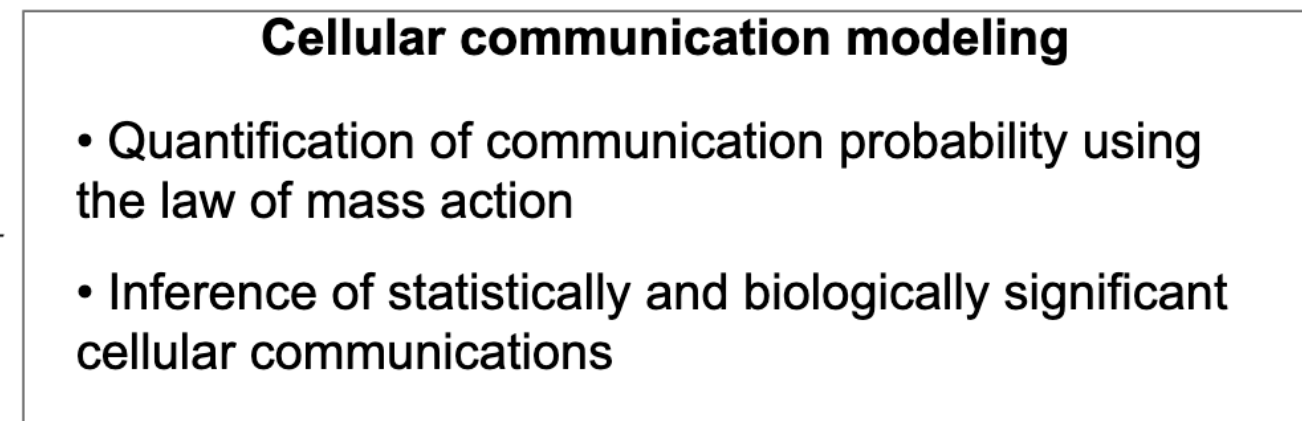
b



d



c



* **Inference of intercellular communications.** a) Identification of differentially expressed signaling genes. To infer the cell state-specific communications, we first identified differentially expressed signaling genes across all cell groups within a given scRNA-seq dataset, using the Wilcoxon rank sum test with the significance level of 0.05.

b) Calculation of ensemble average expression. To account for the noise effects, we calculated the ensemble average expression of signaling genes in a given cell group using a statistically robust mean method:

$$EM = \frac{1}{2}Q_2 + \frac{1}{4}(Q_1 + Q_3) \quad (1)$$

where Q_1 , Q_2 , and Q_3 is the first, second and third quartile of the expression levels of a signaling gene in a cell group.

c) Calculation of intercellular communication probability. We modeled ligand-receptor mediated signaling interactions using the law of mass action. Since the physical process of ligand-receptor binding involves protein-protein interactions, we used a random walk based network propagation technique^{83,84} to project the gene expression profiles onto a high-confidence experimentally validated protein-protein network from STRINGdb^{83,85}. Based on the projected ligand and receptor profiles, the communication probability $P_{i,j}$ from cell groups i to j for a particular ligand-receptor pair k was modeled by:

$$P_{i,j}^k = \frac{L_i R_j}{K_h + L_i R_j} \times \left(1 + \frac{AG_i}{K_h + AG_i}\right) \cdot \left(1 + \frac{AG_j}{K_h + AG_j}\right) \times \frac{K_h}{K_h + AN_i} \cdot \frac{K_h}{K_h + AN_j} \times \frac{n_i n_j}{n^2}, \quad (2)$$

$$L_i = \sqrt[m_1]{L_{i,1} \cdots L_{i,m_1}}, R_j = \sqrt[m_2]{R_{j,1} \cdots R_{j,m_2}} \cdot \frac{1 + RA_j}{1 + RI_j}.$$

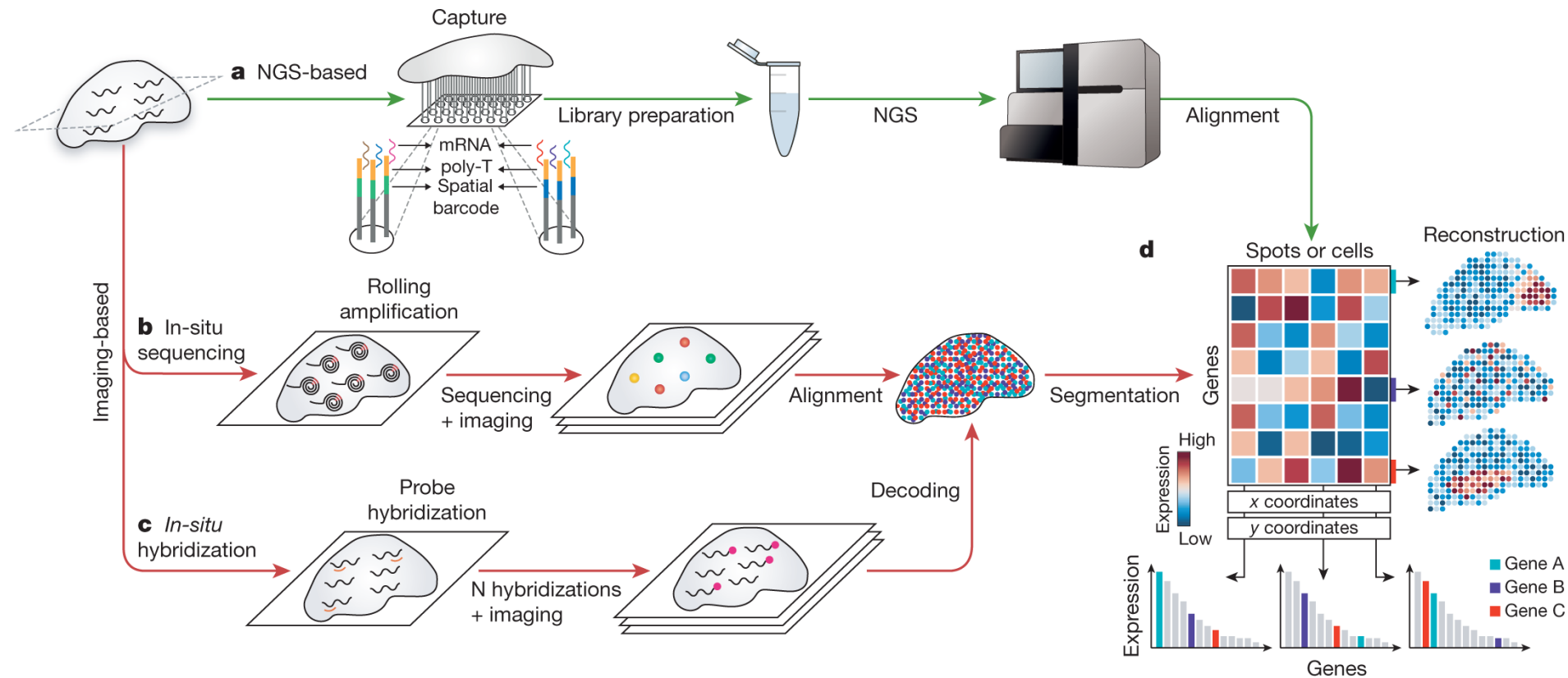
Here L_i and R_j represent the expression level of ligand L and receptor R in cell group i and cell group j , respectively. The expression level of ligand L with m_1 subunits (i.e., $L_{i,1}, \dots, L_{i,m_1}$) was approximated by their geometric mean, implying that the zero expression of any subunit leads to an inactive ligand. Similarly, we computed the expression level of receptor R with m_2 subunits. In addition, co-stimulatory and co-inhibitory membrane-bound receptors are capable of modulating signaling via the control of receptor activation⁸⁶. For the ligand-receptor pair with multiple co-stimulatory receptors, we computed the average expression of these co-stimulatory receptors (denoted by RA) and then used a linear function to model the positive modulation of the receptor expression. For each ligand-receptor pair with multiple co-inhibitory receptors, we modeled them using the same approach. A Hill function was used to model the interactions between L and R with a parameter K_h whose default value was set to be 0.5 as the input data has a normalized range from 0 to 1. The extracellular agonists and antagonists from both sender and receiver cells are able to directly or indirectly modulate the ligand-receptor interaction⁸⁶. For the ligand-receptor pair with multiple soluble agonists, we computed the average expression of these agonists (denoted by AG) and then used a Hill function to model the positive modulation of the ligand-receptor interaction. For the ligand-receptor pair with multiple soluble antagonists, we modeled them using the same approach. The effect of cell proportion in each cell group was also included in the probability calculation when analyzing unsorted single-cell transcriptomes, where n_i and n_j are the numbers of cells in cell groups i and j , respectively, and n is the total number of cells in a given dataset. Together, the communication probabilities among all pairs of cell groups across all pairs of ligand-receptor were represented by a three-dimensional array $\mathbf{P}$ ($K \times K \times N$), where K is the number of cell groups and N is the number of ligand-receptor pairs or signaling pathways. The communication probability of a signaling pathway was computed by summarizing the probabilities of its associated ligand-receptor pairs. It should be noted that we did not perform normalization along the second dimension of $\mathbf{P}$ such that $\sum P_{i,j}^k = 1$ because the normalized data are not suitable for comparing the communication probability between different cell groups across multiple signaling pathways. The communication probability here only represents the interaction strength and is not exactly a probability.

d) Identification of statistically significant intercellular communications. The significant interactions between two cell groups are identified using a permutation test by randomly permuting the group labels of cells, and then recalculating the communication probability $P_{i,j}$ between cell group i and cell group j through a pair of ligand L and receptor R . The p -value of each $P_{i,j}$ is computed by:

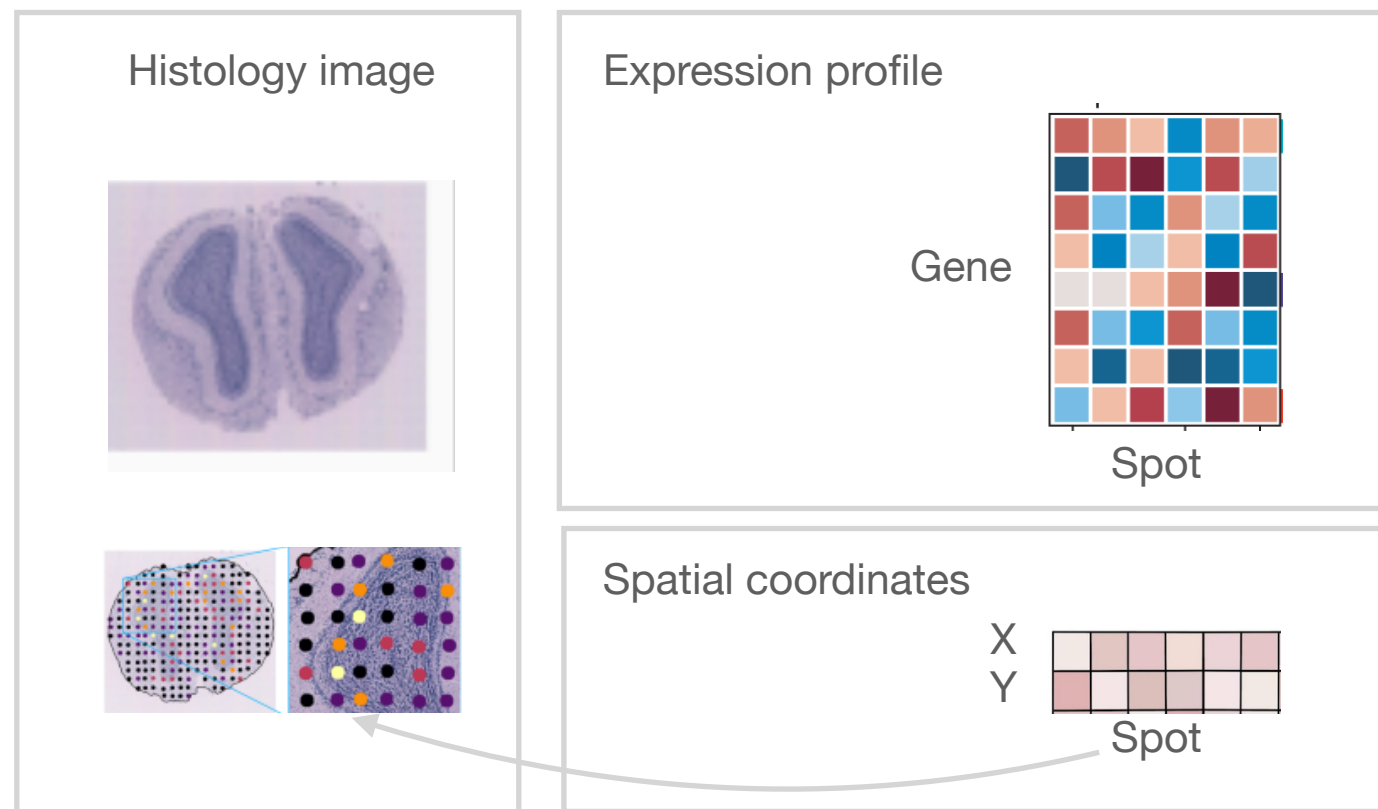
$$p\text{-value} = \frac{\left\{ \#m | P_{i,j}^{(m)} \leq P_{i,j}, m = 1, 2, \dots, M \right\}}{M} \quad (3)$$

where the probability $P_{i,j}^{(m)}$ is the communication probability for the m -th permutation. M is the total number of permutations ($M = 100$ by default). The interactions with p -value < 0.05 are considered significant.

Spatial Transcriptome technology



Spatial Transcriptome Data



COMMOT

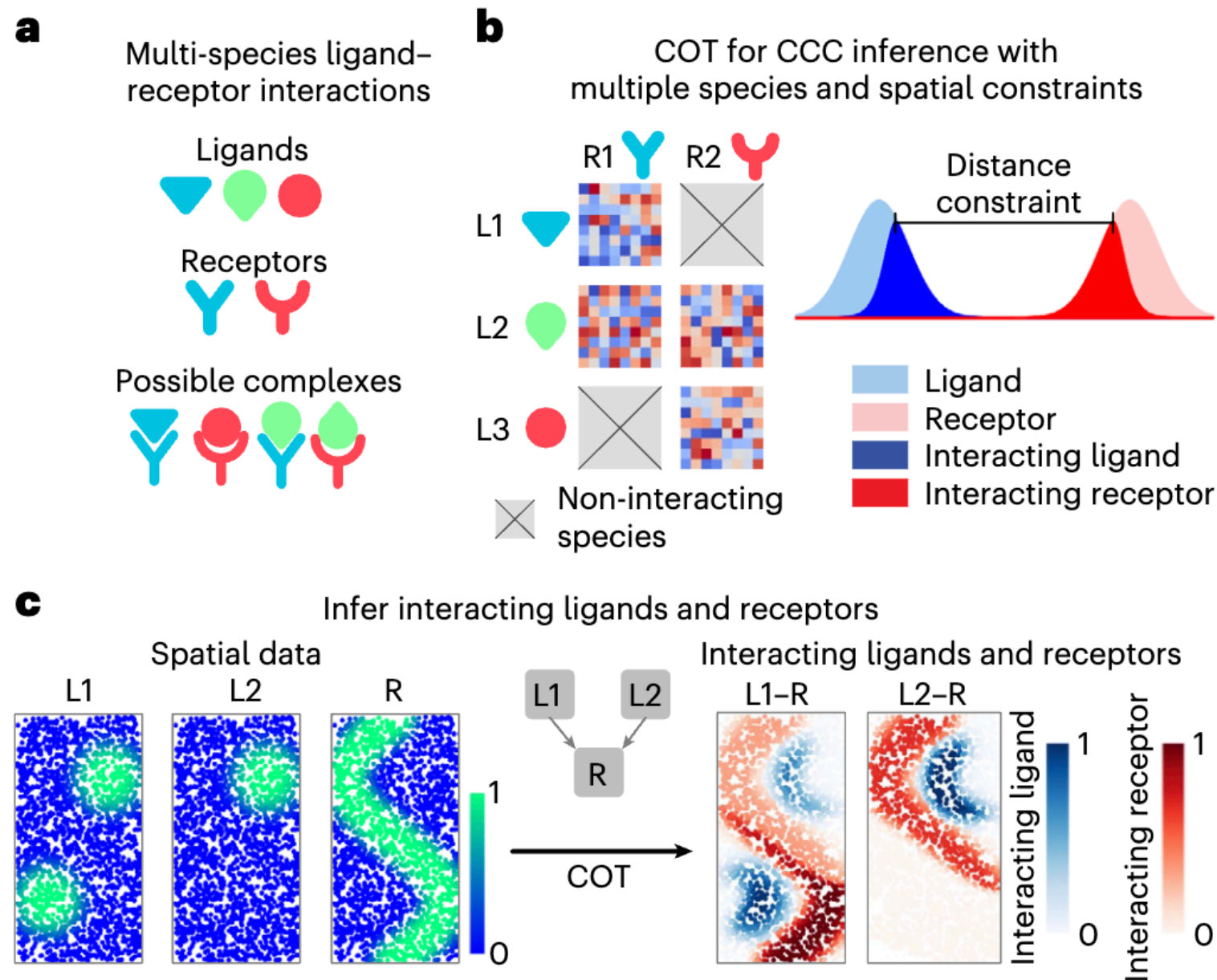


Fig. 1 | Overview of COMMOT. **a**, COMMOT infers CCC in space while considering the competition between different ligand and receptor species. **b**, Collective optimal transport (COT) infers CCC in space by introducing multi-species distributions and enforcing limited spatial ranges. **c**, An example of inferring CCC for spatial distributions of ligand-receptor complexes from spatial distributions of the ligands and receptor where two ligand species (L1, L2) compete for one receptor species (R). **d**, Three applications of downstream analysis based on the inferred CCC network between cells or spots. DEG, differentially expressed gene; dir., direction; w.r.t., with respect to.

* For a spatial transcriptomics dataset of n_s spatial locations and a set of n_l ligand species and n_r receptor species, a collective optimal transport problem is formulated as follows:

$$\begin{aligned} \min_{\mathbf{P} \in \Gamma} \sum_{(i,j) \in I} \langle \mathbf{P}_{i,j,\cdot,\cdot}, \mathbf{C}_{(i,j)} \rangle_F + \sum_i F(\boldsymbol{\mu}_i) + \sum_j F(\boldsymbol{\nu}_j), \\ \Gamma = \left\{ \mathbf{P} \in \mathbb{R}_+^{n_l \times n_r \times n_s \times n_s} : \mathbf{P}_{i,j,\cdot,\cdot} = \mathbf{0} \text{ for } (i,j) \notin I, \sum_{j,l} \mathbf{P}_{i,j,k,l} \leq \mathbf{X}_{i,k}^L, \sum_{i,k} \mathbf{P}_{i,j,k,l} \leq \mathbf{X}_{j,l}^R \right\}, \\ \boldsymbol{\mu}_i(k) = \mathbf{X}_{i,k}^L - \sum_{j,l} \mathbf{P}_{i,j,k,l}, \boldsymbol{\nu}_j(l) = \mathbf{X}_{j,l}^R - \sum_{i,k} \mathbf{P}_{i,j,k,l} \end{aligned} \quad (1)$$

where $\mathbf{X}_{i,k}^L$ is the expression level of ligand i on spot k , $\mathbf{X}_{j,l}^R$ is the expression level of receptor j on spot l and F penalizes the untransported mass $\boldsymbol{\mu}_i$ and $\boldsymbol{\nu}_j$. The coupling matrix $\mathbf{P}_{i,j,k,l}$ scores the signaling strength from spot k to spot l through the pair consisting of the ligand i and receptor j for $(i,j) \in I$ where I is the index set of ligand and receptor species that can bind. The cost matrix $\mathbf{C}_{(i,j)}$ is based on the thresholded distance matrix such that its kl -th entry equals $\varphi(\mathbf{D}_{k,l})$ if $\mathbf{D}_{k,l} \leq T_{(i,j)}$ and infinity otherwise, where $\mathbf{D}$ is the Euclidean distance matrix for the distances between the spots, $T_{(i,j)}$ is the spatial limit of signaling through the pair of ligand i and receptor j , and φ is a scaling function, such as square or exponential. When the ligands or receptors contain heteromeric units, the minimum of units is used by default in the package to represent the amount of ligand or receptor. For example, if receptor species j is composed of two subunits, the minimum of them in spot l is used to represent the level of this receptor species $\mathbf{X}_{j,l}^R$.

Collective optimal transport algorithm

To solve the collective optimal transport problem described above, we rewrite the original problem as:

$$\begin{aligned} \min_{\hat{\mathbf{P}}, \hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\nu}} \geq 0} \langle \hat{\mathbf{P}}, \hat{\mathbf{C}} \rangle_F + \epsilon_p H(\hat{\mathbf{P}}) + \epsilon_\mu H(\hat{\boldsymbol{\mu}}) + \epsilon_\nu H(\hat{\boldsymbol{\nu}}) + \rho (\|\hat{\boldsymbol{\mu}}\|_1 + \|\hat{\boldsymbol{\nu}}\|_1), \\ \text{s.t. } \hat{\mathbf{P}} \mathbf{1}^n = \mathbf{a} - \hat{\boldsymbol{\mu}}, \hat{\mathbf{P}}^T \mathbf{1}^m = \mathbf{b} - \hat{\boldsymbol{\nu}} \end{aligned} \quad (2)$$

where $\hat{\mathbf{P}}$ is obtained by reshaping $\mathbf{P}$ such that $\hat{\mathbf{P}}_{(i-1) \times n_s + k, (j-1) \times n_s + l} = \mathbf{P}_{i,j,k,l}$. The cost matrix $\hat{\mathbf{C}}$ is obtained similarly and we set $\hat{\mathbf{C}}_{(i-1) \times n_s + k, (j-1) \times n_s + l} = \infty$ for ligand i and receptor j that cannot bind. The marginal distributions are constructed such that $\mathbf{a}_{(i-1) \times n_s + k} = \mathbf{X}_{i,k}^L$ and $\mathbf{b}_{(j-1) \times n_s + l} = \mathbf{X}_{j,l}^R$. Entropy regularization is added to speed up computation and smooth the result with $H(\mathbf{x}) = \sum_i x_i (\ln(x_i) - 1)$.

When the entropy regularization terms have the same coefficient values, $\epsilon = \epsilon_p = \epsilon_\mu = \epsilon_\nu$, the problem can be efficiently solved with a stabilized Sinkhorn iteration⁷⁰

$$\begin{aligned} \mathbf{f}^{(l+1)} &\leftarrow \epsilon \log \mathbf{a} + \mathbf{f}^{(l)} - \epsilon \log \left(e^{\frac{\mathbf{f}^{(l)}}{\epsilon}} \odot e^{-\frac{\mathbf{c}}{\epsilon}} e^{\frac{\mathbf{g}^{(l)}}{\epsilon}} + e^{\frac{\mathbf{f}^{(l)} - \rho}{\epsilon}} \right), \\ \mathbf{g}^{(l+1)} &\leftarrow \epsilon \log \mathbf{b} + \mathbf{g}^{(l)} - \epsilon \log \left(e^{\frac{\mathbf{g}^{(l)}}{\epsilon}} \odot e^{-\frac{\mathbf{c}^T}{\epsilon}} e^{\frac{\mathbf{f}^{(l+1)}}{\epsilon}} + e^{\frac{\mathbf{g}^{(l)} - \rho}{\epsilon}} \right), \end{aligned} \quad (3)$$

for $l \geq 0$ with arbitrary initial $\mathbf{f}^{(0)}$ and $\mathbf{g}^{(0)}$. The resulting numerical solution to the optimization problem can be constructed by $\hat{\mathbf{P}}^* = e^{(\mathbf{f} \oplus \mathbf{g} - \mathbf{C})/\epsilon}$. The formulation in Eq. (2) solved by the algorithm in Eq. (3) was used to generate the results in this study. The derivation of the algorithm, and that of algorithms for the general case in which the regularization terms have different coefficients, is described in the Supplementary Information.

Optimal transport

Monge problem

- Monge problem seeks for a map that associates to each point x_i a single point y_j , and which must push the mass of α toward the mass of β , namely:

$$\forall j, \quad b_j = \sum_{i: T(x_i)=y_j} a_i$$

- Discrete case:

$$\min_T \sum_i c(x_i, T(x_i)), \quad \text{s.t.} \quad T_{\#}\alpha = \beta$$

- Arbitrary measures:

$$\min_T \int_{\mathcal{X}} c(x, T(x)) d\alpha(x), \quad \text{s.t.} \quad T_{\#}\alpha = \beta$$

Optimal transport distance between General Measures

$$\mathcal{L}(\alpha, \beta, c) \stackrel{\text{def}}{=} \min_{\pi \in \mathcal{U}(\alpha, \beta)} \int_{\mathcal{X} \times \mathcal{Y}} c(x, y) d\pi(x, y). \quad (9)$$

Probability interpretation:

$$\min_{(X, Y)} \{ \mathbb{E}_{(X, Y)}(c(X, Y)), X \sim \alpha, Y \sim \beta \}. \quad (10)$$

Wasserstein Distance

Metric Space $\mathcal{X} = \mathcal{Y}$.

Distance $d(x, y)$ (nonnegative, symmetric, identity, triangle inequality).

Cost $c(x, y) = d(x, y)^p, p \geq 1$.

Wasserstein Distance

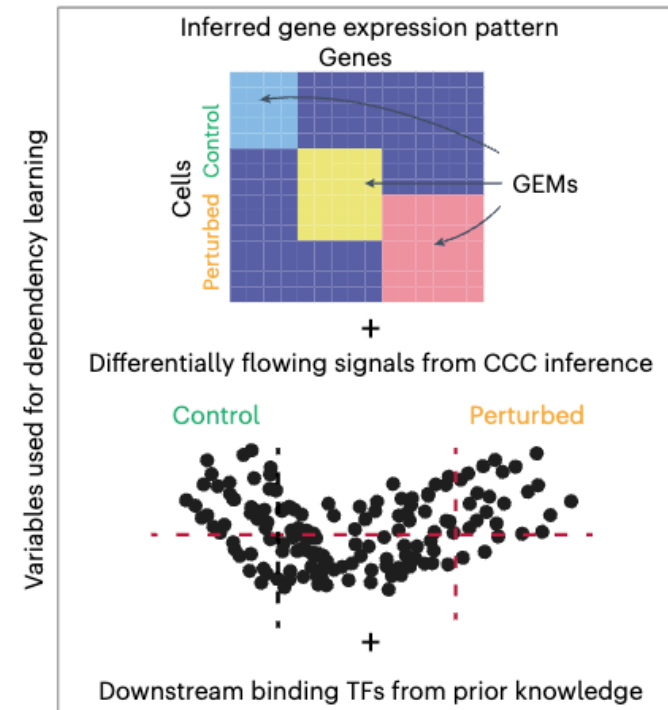
$$\mathcal{W}_p(\alpha, \beta) \stackrel{\text{def}}{=} \mathcal{L}(\alpha, \beta, d^p)^{1/p}. \quad (11)$$

a

FlowSig

Assumed causal model

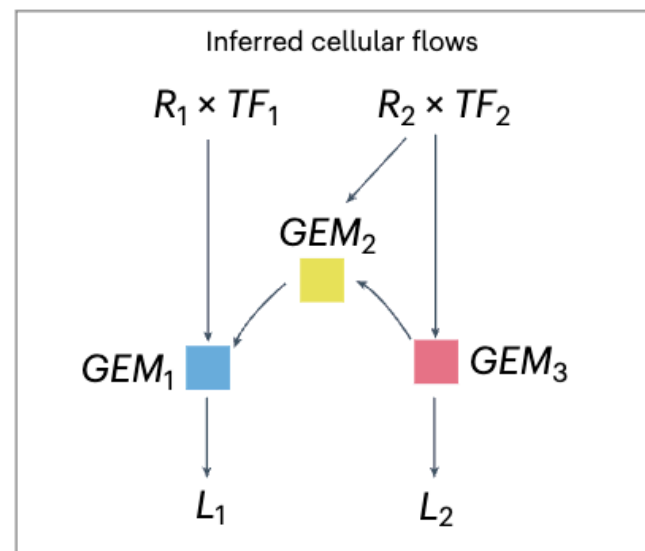
Inflowing signal 1 $\longrightarrow$ GEMs (potential regulatory TFs) $\longrightarrow$ Outflowing signal 2

b Input: scRNA-seq (control versus $n \geq 1$ perturbed)

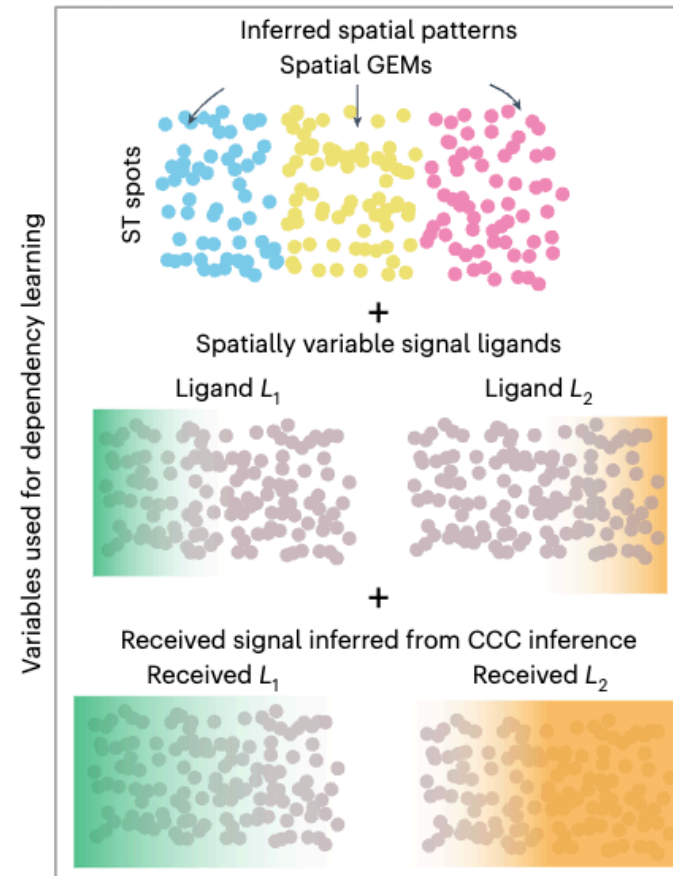
Conditional independence testing
Conditional invariance testing

Graphical model learning

Output

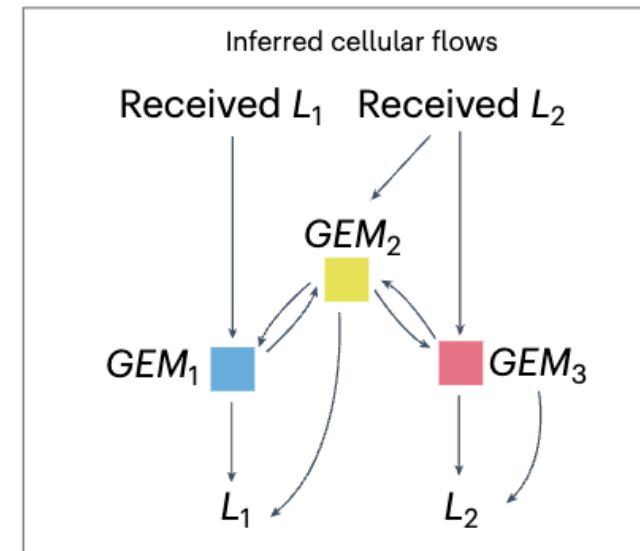


c Input: ST



Conditional independence testing
Graphical model learning

Output



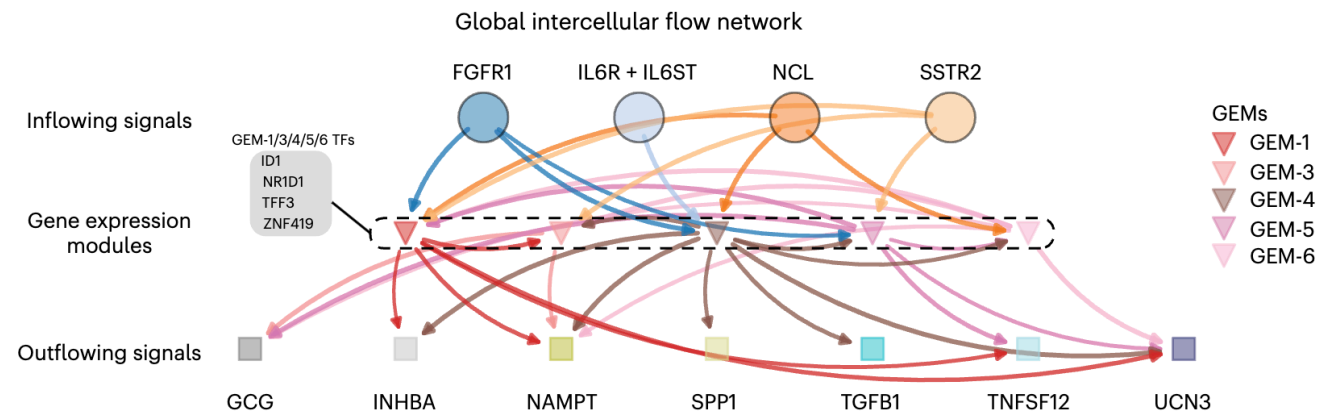
*

FlowSig uses gene expression measurements and output from cell–cell communication inference to learn intercellular flows that describe directed dependencies. These dependencies are oriented from inflowing intercellular signals to intracellular GEMs, which could be individual TFs or cellwise enrichment for correlated gene sets, and from GEMs to outflowing intercellular signals (Fig. 1a). We model the intercellular flows using graphical causal models, where nodes represent the flow variables—inflowing signals, GEMs and outflowing signals—and learn a directed graph using conditional independence testing and the unknown target interventional greedy sparsest permutation algorithm (UT-IGSP)²⁴. Considering that one can use statistical conditional independence relations to infer, at best, a set of equivalent directed acyclic graphs (DAGs) with the same undirected skeleton graph and directed ν -structures (connected node triplets (x, y, z) with the directed edges $x \rightarrow y \leftarrow z$)²⁵, we use UT-IGSP to learn an initial CPDAG, which can contain both directed arcs and undirected edges. We then construct the intercellular flow network by reorienting undirected edges and removing biologically unrealistic arcs so that edges are directed from inflowing signals to GEMs, between GEMs and from GEMs to outflowing signals.

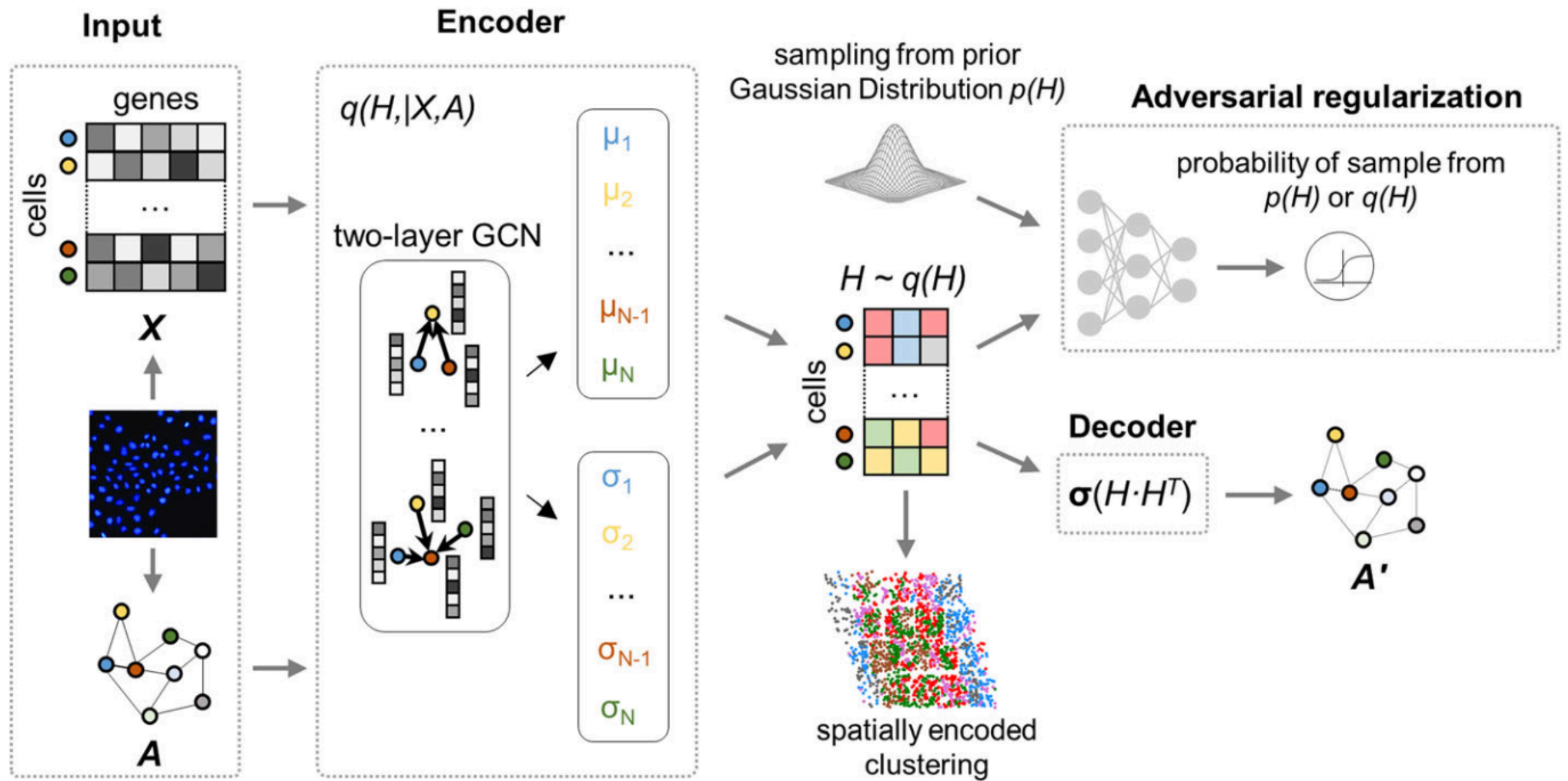
Although the core steps in using FlowSig to analyze non-spatial scRNA-seq and ST data are the same, there are several differences. For non-spatial scRNA-seq data, we must overcome a fundamental issue: it is not possible to directly measure the intercellular signals that each cell received. Therefore, we impose two constraints (Fig. 1b). First, we consider only studies comparing a ‘control’ condition against one or more perturbed conditions, for example, healthy versus diseased. We use the additional information gained from perturbation data through conditional invariance testing to narrow down the set of possible flow graphs, reducing the occurrence of false positive edge discovery. Second, for each ligand–receptor interaction inferred from cell–cell

communication inference, we extract downstream TF targets from the OmniPath database²⁶ to measure signal inflow. Receptor gene expression quantifies the potential for a cell to receive an intercellular signal, and downstream TF expression quantifies the extent to which the cell actually received the signal; we define signal inflow as the product of receptor gene expression and the average expression of downstream TF targets.

ST technologies are currently in their infancy, so there are relatively fewer control versus perturbed ST studies than scRNA-seq studies. However, we can use communication methods such as COMMOT¹⁴ to spatially constrain and measure the amount of inflowing signal more accurately (Fig. 1c). Therefore, FlowSig uses the greedy sparsest algorithm (GSP)²⁷, which does not use perturbation data, to analyze ST data.



DeepLinc



✱

DeepLinc uses neighboring cells with direct contacts as the positive set for learning the transcriptome features related to cell-cell interactions. From a general biological point of view, it is reasonable to assume that in a solid tissue, most of the cells in 2-D could directly contact with 3 or more other cells. From a technical point of view, for the strategies of machine learning, it is critical to minimize the potential false positives in the predefined positive sets for training. Therefore, for each cell, we only used the 3 nearest neighbors to define direct contacts, which we believe is a balanced choice generating enough number of direct contacts for training the DeepLinc pipeline, and at the same time, ensuring few false positives to contaminate the positive training set.

Given a cell adjacency matrix A and a gene expression matrix X , the VGAE learns the embedded features H of the cell graph. A two-layer graph convolutional network (GCN) (250-125) is used as the encoder module.

$$q(H|X, A) = \prod_{i=1}^n q(h_i|X, A)$$

$$q(h_i|X, A) = N(h_i|\mu_i, \text{diag}(\sigma^2))$$

Similar to variational autoencoder, $\mu = GCN_\mu(X, A)$ is the matrix of mean vectors and $\log\sigma = GCN_\sigma(X, A)$. As a typical structure, the GCN is defined as $GCN(X, A) = \tilde{A}ReLU(\tilde{A}XW_0)W_1$ with weight W_i and normalized adjacency matrix $\tilde{A} = D^{-\frac{1}{2}}(A + I)D^{-\frac{1}{2}}$ to realize layer-to-layer information transmission. We use ReLU as the activation function of the first layer instead of the second layer. The purpose of this step is to learn meaningful embedded features supporting the interaction formation by aggregating information of each cell itself and its interacting neighbor cells.

The inner product decoder $p(A|H) = \sigma(H \cdot H^T)$ is employed to reconstruct a new cell interaction matrix, which reports probability scores of the interactions between each pair of single cells.

VGAE has exhibited outstanding performance in learning the embedded features of graph elements [38]. However, for single-cell spatial transcriptome data, a major challenge is its high sparsity and data noise. For such non-ideal situations, the standard variational encoder-decoder structure adopted by VGAE does not make more constraints on latent variables, often resulting in underfitting of the sparse data [63]. Here we implement strong regularization on the latent distribution with the adversarial network.

Specifically, the adversarial module is built on a multilayer perceptron (MLP). The architecture is similar to the discriminator of generative adversarial networks (GAN). The adversarial network has two fully-connected hidden layers with ReLU activation (125–150). The output layer uses sigmoid function to judge whether a latent sample h_i is from a prior Gaussian distribution or from VGAE.

Eventually, the DeepLinc pipeline was optimized to maximize the log-likelihood of cell-cell interaction network. The objective of VGAE is to minimize its cost function by optimizing the evidence lower bound (ELBO):

$$\zeta_{ELBO} = E_{q(H|X, A)}[\log p(A|H)] - D_{KL}(q(H|X, A) || p(H))$$

For the adversarial module, the loss function is binary cross-entropy.

CellNest

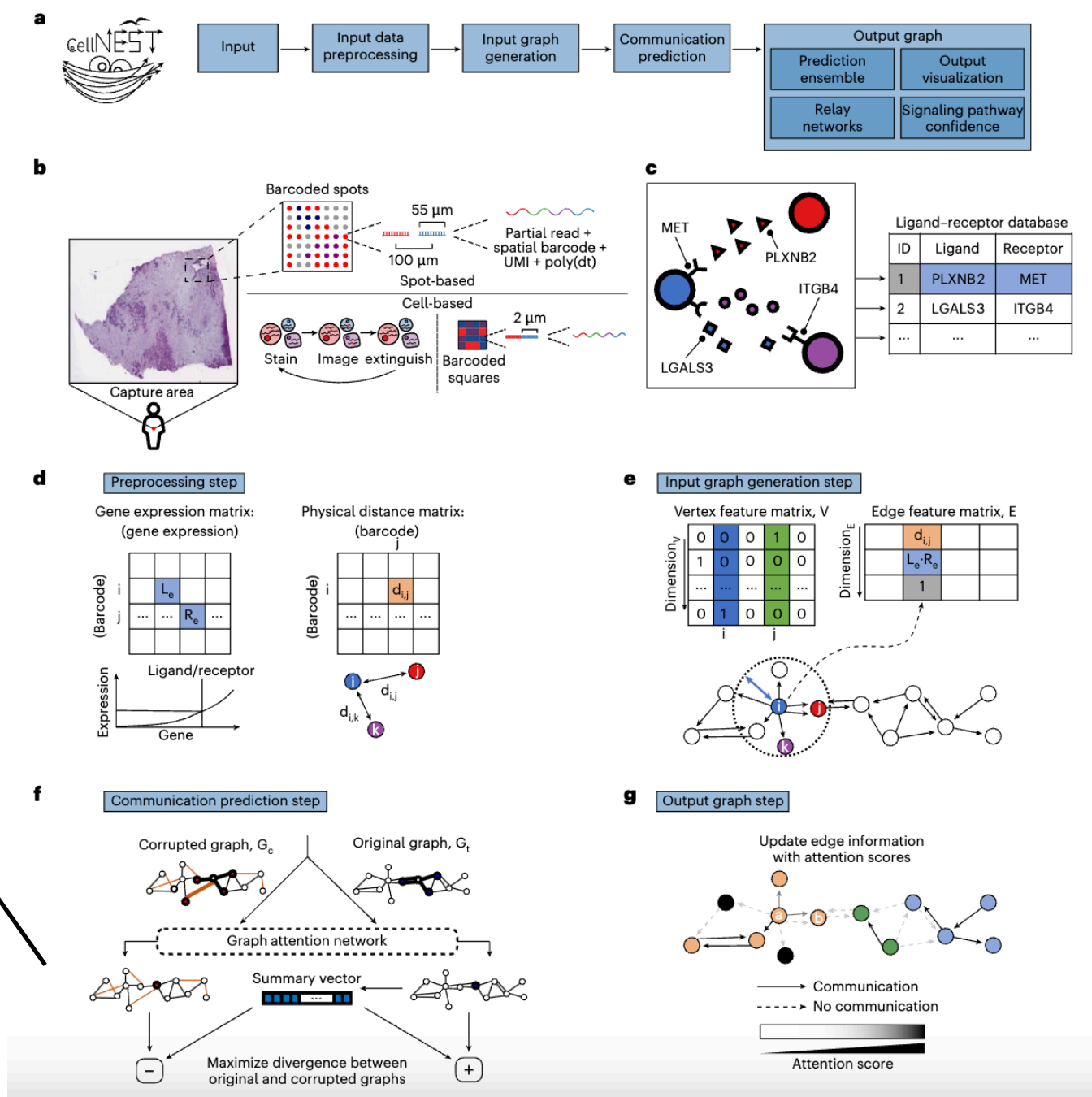


Fig. 1 | Overview of detecting cell-cell communication with CellNest.

a, A high-level flowchart of the main steps of the CellNest method. **b**, Input tissue sample at either spot (for example, Visium; top) or cell resolution (for example, MERFISH, Visium HD; bottom). UMI, unique molecular identifier. **c**, Input ligand-receptor database containing known ligand and cognate receptor pairings. **d**, Preprocessing step, where genes with expression above a threshold percentile are considered active (left). Pairwise Euclidean distances between vertices are stored in a physical distance matrix (right). **e**, Input graph $G = (V, E)$ generation step with V spots or cells as vertices and E edges as neighborhood relations, some of which represent communication (bottom). An input threshold distance is used for the neighborhood formation (blue arrow). From the graph, vertex features are represented as a one-hot vector matrix (top left). The edge feature matrix holds edge feature vectors containing three attributes: pairwise distance, ligand-receptor coexpression score and the ligand-receptor pair

identity from the database in **c**. **f**, Communication prediction step using a GAT encoder through unsupervised contrastive learning with DGI. **g**, Output graph step visualizing edges with the highest attention scores. Attention scores range from 0 (white) to 1 (black), where 1 represents the strongest connections. Lower-scoring edges are removed (dashed lines), resulting in subgraphs of communicating vertices. **h**, Example output showing the flow of communication between tumor-annotated spots (filled squares) with stroma spots (open circles), colored by connected component. **i**, An example CellNest-generated histogram showing the frequency of communication through ligand-receptor pairs in the top 20% highest-scoring attention edges. Colors in the histogram correspond to connected components in **h**. For instance, the most abundant communication, labeled as *FNI-RPSA*, is found primarily in the blue region. Altogether, CellNest offers a high-resolution approach for detecting the strength and location of cell-cell communication in tissues.

The traditional GAT model requires ground-truth data for training an encoder, but this information is unknown from spatial transcriptomic data. We instead chose to implement unsupervised training through DGI³³, a contrastive learning approach (Fig. 1f). DGI compares encoder weights derived from the observed network with encoder weights from a ‘corrupted’ network of randomly shuffled and permuted vertices and edges. DGI maximizes weights from the observed network while penalizing weights from the corrupted network. As the model converges, CellNEST assigns higher attention scores to stronger neighborhood relations (Fig. 1g). We use these attention scores to represent communication strength. To retain the most probable intercellular signals, we filter edges, retaining the top 20% of highest-scoring attention edges by default (Supplementary Note 2).



THANK YOU